Codity — Distributed Job Scheduler

Intern Technical Assignment Submission

Author: Surianandhan **Date:** 2026-08-24 **Repository:** https://github.com/Surianandhan/Codity

---

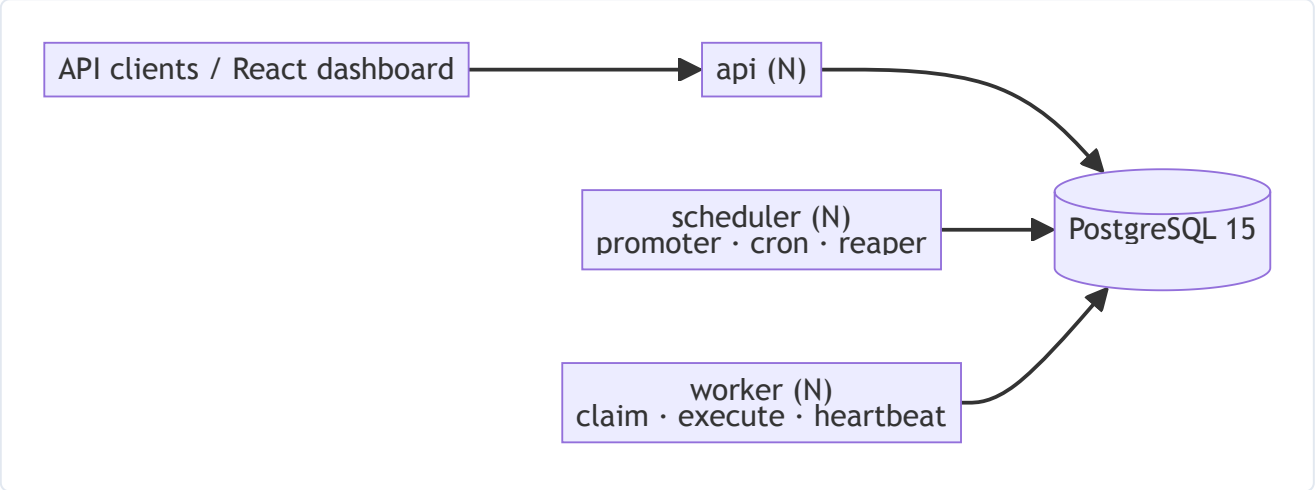
A production-inspired distributed job scheduling platform on PostgreSQL 15 — no Redis, no broker, no Docker. Jobs are claimed with `SELECT ... FOR UPDATE SKIP LOCKED`; reliability is enforced with database constraints, not application discipline alone.

This document combines the project's own documentation — written and adversarially reviewed before a line of code was written — with evidence from running the live system, including one genuine `kill -9` recovery. It is a complete written submission; the linked repository holds the executable source.

Overview & Setup

A production-inspired background job platform: submit jobs over a REST API, and a fleet of worker processes claims and executes them across machines. Jobs can be immediate, delayed, scheduled, recurring (cron), or submitted in batches. Failures retry with jittered backoff, permanent failures land in a dead letter queue an operator can inspect and replay, and a worker that dies mid-job has its work reclaimed and finished by another — **without ever running the job twice against the same attempt**.

The architecture in three sentences. Three stateless process types — `api`, `worker`, `scheduler` — share nothing but a single PostgreSQL 15 database; there is no Redis, no Celery, and no broker. Workers claim jobs with a single `SELECT … FOR UPDATE SKIP LOCKED` statement that transitions the job and opens its execution row atomically, so K workers partition the ready set with no coordination and no blocking. Every ownership change bumps a `lease_epoch` fencing token, so a worker that stalls and is reclaimed can never commit a stale result — which is what makes crash recovery safe rather than merely likely.



⚠️ Nothing executes without a running worker

This is the single most likely way to conclude the system is broken when it is working exactly as designed.

The API **accepts** jobs and returns `201`. It never executes them. A job stays in `status: "queued"` until a `worker` process claims it. If you POST a job with no worker running, you will see a job that never moves — that is correct behaviour, not a bug.

You need **three backend processes**, in three terminals: `make api`, `make worker`, `make scheduler`. The dashboard is a fourth terminal and is optional — every step below can also be driven from Swagger.

The scheduler is not optional either. Without it, *immediate* jobs still flow perfectly while every *delayed* job, every *cron* occurrence, and **every backoff retry** stalls silently. That is the most confusing failure the system can have, which is why the scheduler's last-tick age is reported at `GET /api/v1/system/status` and shown on the dashboard.

Quickstart — no Docker required

macOS with Homebrew, PostgreSQL 15, Python 3.13, and `uv`. Docker is not used anywhere in this project, including in the tests.

Start PostgreSQL:

```
brew services start postgresql@15
```

Create the two databases (the second is for the test suite):

```
createdb codity && createdb codity_test
```

Install Python dependencies:

```
cd backend && uv sync
```

Apply migrations:

```
cd backend && uv run alembic upgrade head
```

Seed a demo organization, project, queues and handlers:

```
cd backend && uv run python scripts/seed.py
```

The seed prints an **organization id** and a login. Keep the org id — the worker needs it, because a worker connects to Postgres directly and must be told which tenant it serves.

Run the system — three backend processes plus the dashboard

API:

```
cd backend && uv run uvicorn app.main:app --reload --port 8000
```

Worker — **replace the placeholder with the org id the seed printed**; there is no environment variable to fall back on, and pasting this line unedited passes an empty `--org`:

```
cd backend && uv run python -m app.worker.main \
  --org PASTE-ORG-ID-FROM-SEED --name worker-1 --concurrency 4
```

Do not export `CODITY_ORG_ID` and expect the worker to read it. Nothing reads it, and worse, `config.py` uses `env_prefix="CODITY_"` with `extra="forbid"`, so an unrecognised `CODITY_*` name in the environment raises a `ValidationError` that takes down the API, the worker and the scheduler alike. Use `make worker`, which resolves the org id from the database for you.

Scheduler:

```
cd backend && uv run python -m app.scheduler.main
```

Dashboard:

```
cd frontend && npm install && npm run dev
```

Then open http://localhost:5173 for the dashboard and http://localhost:8000/docs for Swagger.

Or use the Makefile

Every command above has a target. `make` on its own lists them.

```
make setup && make migrate && make seed
```

`make api` · `make worker` · `make worker2` · `make scheduler` · `make demo` · `make test` · `make check`

`make worker` resolves the org id from the local database automatically; override with `make worker ORG=<uuid>`.

See it work

`docs/GRADER_WALKTHROUGH.md` **is a numbered ~10 minute path** through every capability, with each step annotated by what it demonstrates. The short version:

```
make demo
```

500 jobs across 3 queues with a handler that fails ~20% of the time. The throughput chart moves, failures retry with visibly increasing gaps, and some jobs reach the DLQ. At the end it prints an invariant block:

```
duplicate first-attempt executions = 0
terminal == enqueued
```

Then start a second worker, re-run the load, and send `SIGKILL` to the first one mid-flight. Within one lease period the reaper reclaims its jobs, worker-2 finishes them, and the job timeline shows

```
attempt 1 · claimed by worker-1 → lease expired (lost)
attempt 2 · claimed by worker-2 → running → completed
```

with the invariant block still clean. That is the whole design in one screen.

Verified live. This is not a description of what the code should do. The `kill -9` run was performed: a worker was `SIGKILL`ed mid-job, the reaper closed the orphaned execution as `lost` with `error_class='LeaseExpired'`, and a different worker claimed the requeued job at the next epoch and completed it. The timeline above is what that run produced.

Documentation

Document	What is in it
`docs/ARCHITECTURE.md`	Process topology, architecture diagram, module layout, the layering rule, observability
`docs/DATABASE.md`	ER diagram, table-by-table rationale, every index and the query it serves, the status state machine
`docs/DESIGN_DECISIONS.md`	ADR-lite: 17 decisions, each with the option rejected and why
`docs/API.md`	Endpoints, auth, error envelope, keyset pagination, idempotency
`docs/SCHEMA_NAMES.md`	The canonical data dictionary. Every other document cites it
`docs/TESTING.md`	What is tested, why every test runs against committed sessions, and what is deliberately not tested
`docs/GRADER_WALKTHROUGH.md`	The 10-minute tour

Diagrams are inline Mermaid, so they render on GitHub and change in the same diff as the code they describe.

Reliability, in one table

Failure	Detection	Recovery	Residual risk
Worker killed mid-job	`lease_expires_at < now()`	Reaper closes the execution as `lost`, requeues the job, bumps the epoch	Side effects from the dead attempt already happened — handler idempotency covers it
Worker slow, stopped, or partitioned	Heartbeat stops	Reaper reclaims; the zombie's writes are rejected by the epoch fence	The zombie's external side effect cannot be un-sent
Database connection lost mid-job	asyncpg raises	Job never transitions; the lease expires; the reaper requeues	One extra attempt consumed only if the job had started
Job exceeds `timeout_ms`	`asyncio.wait_for`	Marked `timed_out`, counts as a failed attempt, backs off	A blocking handler runs to completion in its thread; bounded by `timeout_ms < lease_seconds × 1000`
Poison-pill job	Attempt budget exhausts	Dead-lettered with an error fingerprint; the DLQ groups by fingerprint	None
Two schedulers race a cron occurrence	—	`UNIQUE (schedule_id, scheduled_for)`	None
Clock skew between nodes	—	All time comes from `now()` **inside Postgres**; no process compares its own clock to a lease	None by construction

Delivery is at-least-once, and the docs say so. Exactly-once across a process boundary is impossible — a worker cannot atomically commit "job done" to Postgres and "email sent" to a third-party API. What *is* guaranteed: at most one worker holds a valid lease at any instant, at most one execution can record a result for a given `lease_epoch`, and handlers are given a written idempotency contract. See ADR-005.

Deliberately deferred

Each of these was designed, costed, and cut on purpose. The design and the migration path for each is written down, so none of them is a surprise later.

Deferred	Why	Where the design lives
PostgreSQL RLS	A *second* enforcement layer for something already enforced by composite foreign keys. Roughly a day, and every fixture or migration that forgets to set the GUC becomes an opaque "zero rows, no error" debugging session. That day went to the reliability core instead.	ADR-013 — including the exact policy DDL
`api_keys` (service-to-service auth)	Nothing consumes it yet. The dashboard uses JWT, and workers connect to Postgres directly rather than through the API, so an API key would be an unused table with an unused rotation story. It becomes necessary the moment a third-party service posts jobs.	`docs/API.md` §2
Four-tier RBAC (`owner/admin/operator/viewer`)	RBAC is listed as a *bonus*. A four-tier ladder puts a bonus on the critical path of every endpoint and multiplies the auth test matrix by four. Two roles (`owner`, `member`) ship; the ladder slots into the same dependency as `require_role(min_role)` with no schema change beyond widening a CHECK.	ADR-014
WebSocket live updates	Also a listed *bonus*. Reconnect-with-backoff, auth on upgrade, resubscribe, server-side fanout, and missed-event reconciliation are a lot of subtle code. Polling is correct by construction — every poll re-reads the truth — and its cost is bounded: terminal jobs stop polling, hidden tabs pause. The polling hooks are the seam if the transport is swapped later.	ADR-015
Table partitioning	Monthly declarative partitioning of `job_logs` and `job_executions` turns retention into a partition detach. At this volume the batched sweep is measurably sufficient, and partitioning would add migration complexity with no observable benefit.	`docs/DATABASE.md` §7
Queue sharding	Claims are serialised *per queue* by the `FOR NO KEY UPDATE` lock that makes `max_concurrency` exact. Hashing `queue_id` into shards removes that serialisation — worth doing when one queue needs more claim throughput than one short statement provides, and not before.	ADR-003
Duration percentiles	`avg` and `max` over `job_executions.duration_ms` are what the summary returns, and both ignore NULL rather than coalescing to zero — an attempt reaped before it started is unmeasured, not instantaneous. True percentiles need `percentile_cont` over the same column,	`docs/ARCHITECTURE.md` §5

Deferred	Why	Where the design lives
	which is a wider scan than the stat cards justify at this volume.	
A pre-aggregated metrics rollup	Summary and throughput are computed by aggregating `jobs` and `job_executions` **directly**, with empty buckets gap-filled by `generate_series` so an outage renders as a gap rather than as zero. No rollup table sits in front of them and none is needed at this volume, where the scans are bounded by retention. A rollup becomes worth building at the volume where scanning those tables per request stops being cheap — and it would buy that speed with a second source of truth that can drift, a backfill, and its own tests.	`docs/ARCHITECTURE.md` §5
Removing the vestigial `queue_stats_minute`	The table is created by migration `eb052146b351`, written only by `scripts/seed.py`, swept by the retention loop, and **read by nothing**. It is a leftover of the rollup design above. Dropping it is a migration nobody has written yet; it is dead weight, not a hidden feature.	`docs/DATABASE.md` §9
`worker_queue_assignments`	Queue subscription is not implemented at all: a worker serves every non-paused queue in its organization, and nothing persists a worker→queue mapping. The database therefore cannot answer "which workers serve this queue", so the dashboard cannot distinguish an unserved queue from an idle one. A three-column table plus an upsert on worker boot closes it.	`docs/ARCHITECTURE.md` §1
An `idempotency_keys` table	`Idempotency-Key` ships without it: the job row **is** the idempotency record, uniqueness is `ux_jobs_live_idempotency`, and the request fingerprint is *recomputed* from the persisted job rather than stored. One fidelity gap follows and is stated rather than hidden — a `delayed` job's timing is not exactly reconstructible, so it is excluded from the fingerprint. The table buys back exact body hashing and a stored response.	`docs/API.md` §4
A status-transition trigger	The schema contains **no triggers at all**. Every transition is a guarded `UPDATE` whose `WHERE` names the source status and whose rowcount every caller checks — one enforcement point, and the one the concurrency tests actually exercise. `LEGAL_TRANSITIONS` in `app/domain/enums.py` is the diagram as data, currently referenced by nothing; it is the	`docs/DATABASE.md` §5

Deferred	Why	Where the design lives
	edge set a `BEFORE UPDATE` trigger and its test would assert against.	
The `TenantSession` loader hook	What shipped is hand-written `organization_id` filtering in every router. There is no `TenantSession` and no `with_loader_criteria` hook — a router that forgets its predicate would read across tenants and nothing would catch it. The composite FKs still make a forged cross-tenant *row* unrepresentable, which is the stronger half and is real. Stated plainly rather than implied.	ADR-013
`/auth/refresh`, `/auth/logout`, token rotation	`refresh_tokens` and its `replaced_by_jti` column are in the schema; nothing writes them, and neither endpoint is built. Login returns the refresh token in the JSON body — there is no httpOnly cookie anywhere in the codebase — so an expired access token means logging in again. Rotation with reuse detection is a design, not code.	`docs/API.md` §2
Org, project and queue admin CRUD	Only the routes the dashboard and the walkthrough exercise are built: create and list projects and queues, pause/resume, stats. There is no `GET` / `PATCH` on an org, no membership endpoint, no retry-policy endpoint, and no `GET` / `PATCH` / `DELETE` on a single project or queue. Each is ordinary CRUD over tables that already exist; none of it demonstrates anything the reliability core does not.	`docs/API.md` §1
Bulk DLQ replay	Replay is per-job, from the job detail screen, and it is a real endpoint (`POST /dlq/{entry_id}/replay`). Selecting rows in the explorer and replaying them as a batch is a UI affordance and a partial-failure story that is not built.	`docs/API.md` §1
CI	There is no `.github/` , no workflow file, and no pipeline of any kind. `make check` — ruff, mypy strict, and the full suite — is the only gate, and it is run by hand before every commit. A workflow that runs that same target is a ten-line file; nothing in the repository depends on its absence, and no document should be read as claiming one exists.	—
A repositories layer	`app/repositories/` exists and is **empty** — a zero-byte `__init__.py` . Routers issue `select()` directly, so read logic shared between routers is currently duplication. Extracting it is the natural next refactor; treating	`docs/ARCHITECTURE.md` §3

Deferred	Why	Where the design lives
	the tree as three layers is the accurate reading until then.	

An exactly-once *delivery* guarantee is not on this list, because it is not deferred — it is impossible, and claiming it would be the least honest thing in the repository.

Development

```
make check
```

ruff + mypy (strict) + the full test suite. Run it before every commit.

```
make test-concurrency
```

Just the concurrency tests, which run against real committed sessions — one connection per simulated worker, because `SKIP LOCKED` semantics do not exist inside a single transaction.

Tests run against a real local `codity_test` database. **No testcontainers**, because Docker is not available here and because the behaviour under test is genuine PostgreSQL locking, which does not survive being mocked. Details in `docs/TESTING.md`.

Troubleshooting

A job stays `queued` and never runs. No worker is running, or the worker is subscribed to a different queue or a different organization. Check the worker terminal for claim logs, and check `GET /api/v1/orgs/{org_id}/workers` — a queue with no assigned worker is indistinguishable from an idle queue unless you look.

Delayed, scheduled, or retried jobs never become `queued`, but immediate jobs work fine. The scheduler is not running. Confirm with `GET /api/v1/system/status`: `promoter.last_run_at` will be stale. Start it with `make scheduler`.

`createdb: database "codity" already exists` Harmless — it already exists. `make setup` tolerates this.

`connection refused` on port 5432. PostgreSQL is not running: `brew services start postgresql@15`. Confirm with `psql -l`.

`FATAL: database "codity_test" does not exist` when running tests. Run `createdb codity_test`, or `make setup`.

`alembic upgrade head` reports a conflicting head. Two migration branches exist. `uv run alembic heads` shows them; they need a merge revision.

Everything is slow and jobs claim in bursts. Expected under a low `max_concurrency` — the cap is exact, and claims for one queue are serialised by design. Raise `max_concurrency`, or spread the load across more queues (ADR-003).

A job says `dead_letter` but the handler looks correct. Check `last_error_class`. `LeaseExpired` means the lease ran out rather than the handler failing — usually a blocking handler that did not use `asyncio.to_thread` and froze the event loop, which starves the heartbeat for *every* job on that worker.

The dashboard shows 401 after a page reload. The access token is short-lived and there is no refresh flow — `/auth/refresh` is not implemented, and nothing is stored in a cookie. Log in again. If login itself fails from the browser but works from `curl`, the API and the dashboard are on origins the CORS config does not allow — check `CODITY_CORS_ORIGINS`.

A test that spawns concurrent workers passes suspiciously. Check that it opened one session per simulated worker. Two coroutines sharing a session are one worker with extra steps, and `SKIP LOCKED` has nothing to skip inside a single transaction — the test passes for the wrong reason. See `docs/TESTING.md` §5.

Stack

Python 3.13 · FastAPI · Pydantic v2 · SQLAlchemy 2.0 (async) · Alembic · asyncpg · PostgreSQL 15 · React + TypeScript + Vite + TanStack Query + Tailwind · `uv`

PostgreSQL is the only infrastructure dependency. That is the point.

Architecture

Codity is a distributed job scheduler whose only infrastructure dependency is PostgreSQL 15. Jobs are submitted over a REST API and executed by a fleet of worker processes that claim work with `SELECT … FOR UPDATE SKIP LOCKED`. There is no Redis, no Celery, and no broker.

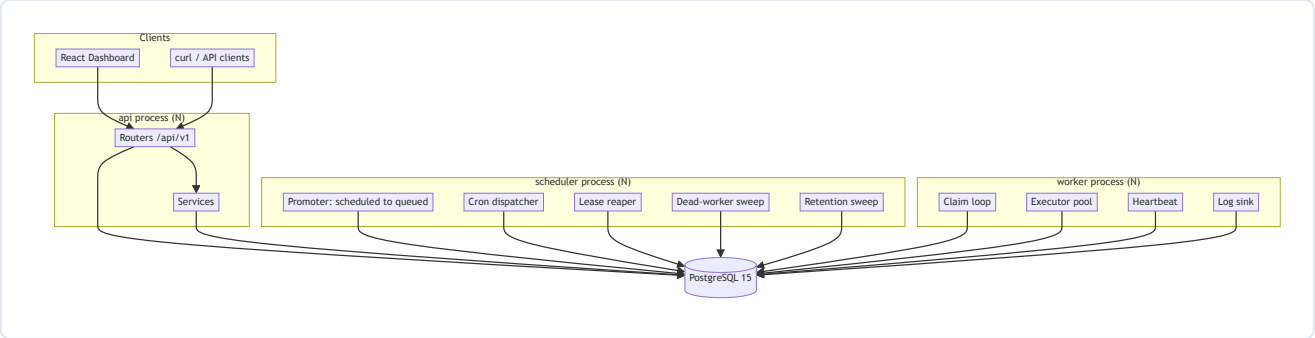
Names used below are defined once in `SCHEMA_NAMES.md`. Rationale for each choice is in `DESIGN_DECISIONS.md`.

---

1. Process topology

Three process types. All are stateless, all speak only to Postgres, and all can be scaled to N replicas without coordination.

Process	Owns	Scaling
`api`	FastAPI/uvicorn. HTTP, auth, validation, job creation, dashboard reads. **Never claims jobs.**	N replicas, stateless
`worker`	Claim → execute → complete/fail. Heartbeats. Graceful drain.	N replicas; this is the throughput knob
`scheduler`	Promoter, cron dispatcher, lease reaper, dead-worker sweep, retention sweep.	N replicas; correctness does not depend on N=1



Every arrow terminates at Postgres. The processes never talk to each other. That is the whole coordination story: **all shared state, all mutual exclusion, and all ordering live in the database**, which is why any of the three can be killed, restarted, or duplicated without a protocol to renegotiate.

Why the scheduler is its own process

It is not an API background task, because that ties job liveness to HTTP traffic — a system with no requests stops promoting delayed jobs — and it double-fires on every API replica.

It is not a leader-elected worker role, because leader election is a distributed-systems problem this system does not need to solve. Correctness under N schedulers comes from the database:

- **Cron double-promotion is impossible** — `UNIQUE (schedule_id, scheduled_for)` on `jobs`.
- **Reaper/promoter overlap is harmless** — every statement is `FOR UPDATE SKIP LOCKED`, so two schedulers *partition* the work instead of colliding.

A `pg_try_advisory_lock` is used only to stop redundant ticks burning CPU. If it is never acquired, the system is still correct. That is the test of whether a lock is load-bearing: remove it and see whether an invariant breaks. This one does not.

Worker identity and queue subscription

A worker connects to Postgres directly, so it must be told which tenant it serves: **`--org <uuid>` is a required CLI argument**, and it is the only way to pass the tenant. There is deliberately no `CODITY_ORG_ID` environment variable — and exporting one would be actively harmful, because `config.py` uses `env_prefix="CODITY_"` with `extra="forbid"`, so any unrecognised `CODITY_*` name in the environment raises a `ValidationError` that takes down the API, the worker and the scheduler alike. The worker's full argument list is `--org`, `--name` and `--concurrency`; there is **no `--queues` flag**. A worker serves every non-paused queue in its organization, choosing between them each polling cycle in weighted-random order by `queues.priority`. Per-queue subscription is not implemented — see the deferred list in the README.

On boot the worker upserts its `workers` row keyed on `(organization_id, name)` where the default name is `{hostname}-{pid}`. A retention rule deletes `workers` rows in status `dead` / `stopped` older than 24h, so restarts do not leave the fleet screen full of corpses.

Queue subscription is **not persisted**: there is no `worker_queue_assignments` table, so the database cannot answer "which workers serve this queue". The dashboard therefore cannot distinguish a queue with no workers from an idle queue — see the deferred list in `../README.md`.

Per polling cycle the worker iterates its queues in **weighted-random order by `queues.priority`**, issuing one claim statement per queue and capping each queue's share at `ceil(batch_size / n_queues)`. Weighted-random rather than strict ordering is what stops a busy high-priority queue starving a low-priority one — and both properties are testable.

---

2. Module layout

```
Cody/
├── backend/
│   ├── app/
│   │   ├── main.py                 # FastAPI app factory
│   │   ├── config.py               # pydantic-settings, CODITY_ prefix
│   │   ├── domain/                 # FastAPI-free: enums.py, errors.py, backoff.py
│   │   ├── db/
│   │   │   ├── session.py
│   │   │   ├── models/             # base, tenancy, scheduling, execution, observability
│   │   │   └── sql/                # raw .sql for the hot path
│   │   │       ├── lock_queue.sql  # step 1 of the claim -- see ADR-003
│   │   │       ├── claim_jobs.sql  # step 2
│   │   │       ├── start_job.sql
│   │   │       ├── complete_job.sql
│   │   │       ├── fail_job.sql
│   │   │       ├── reap_leases.sql
│   │   │       ├── promote_due.sql
│   │   │       └── heartbeat.sql
│   │   ├── services/               # jobs, claim, idempotency, reliability, security
│   │   ├── api/
│   │   │   ├── deps.py             # auth, scope resolution
│   │   │   ├── errors.py           # exception handlers, one envelope
│   │   │   ├── pagination.py       # keyset cursors
│   │   │   ├── schemas.py          # every request/response model
│   │   │   ├── middleware/request_context.py
│   │   │   └── routers/            # auth, projects, jobs, schedules, workers, dlq,
│   │   │                           #   metrics, system
│   │   ├── worker/
│   │   │   ├── main.py             # CLI entrypoint (--org is required)
│   │   │   ├── runner.py           # claim loop, executor pool, heartbeat, drain
│   │   │   ├── logsink.py          # the only writer of job_logs
│   │   │   └── handlers/           # demo.echo, demo.sleep, demo.flaky,
│   │   │                           #   demo.always_fail, demo.cpu
│   │   └── scheduler/
│   │       ├── main.py             # process entrypoint
│   │       └── loops.py            # promoter, reaper, cron, dead-worker, retention
│   ├── alembic/versions/           # five revisions, hash-named -- DATABASE.md §9
│   ├── scripts/                    # seed.py, demo_load.py
│   ├── tests/
│   └── pyproject.toml
├── frontend/
├── docs/
└── Makefile
```

The **hot path is raw SQL in `.sql` files**, not ORM expressions. The claim, start, complete, fail, reap, promote and heartbeat statements are each a single multi-CTE statement whose exact shape is the correctness argument. Keeping them as SQL means they are reviewable as SQL, can be pasted into `psql` and `EXPLAIN`ed directly, and cannot be silently restructured by a query-compiler change. The ORM owns CRUD, where its ergonomics pay off and its generated SQL does not matter.

---

3. The layering rule

Three layers, not four:

```
routers → services → models
```

over a FastAPI-free `domain/`.

- `domain/` imports nothing from `app.api`, `app.db`, or `fastapi`. It is pure: enums, errors, backoff arithmetic. (Cron parsing is **not** here — it lives in `scheduler/loops.py`, which is the only caller.)
- `services/` never imports `fastapi`. **This is what lets the worker and the scheduler reuse `services/claim.py` without dragging in the web framework** — the rule exists for that specific reason, not as decoration, and the concurrency tests import the same module the worker does.
- Routers hold no business logic. They do, however, **issue `select()` directly** for reads: there is no repository layer between a router and the ORM.
- Nothing outside `app/api/` raises `HTTPException`. Services raise `DomainError` subclasses, and `app/api/errors.py` is the only place that knows about status codes.

On `app/repositories/`: the package exists and is **empty** — a zero-byte `__init__.py` and nothing else. It is a placeholder for a repository layer that was planned and not built. Read reuse across routers is currently duplication, and extracting it is the natural next refactor; until then, treating the tree as three layers is the accurate reading.

These rules are stated intent, not machine-checked. No test walks the AST asserting them, so nothing stops a future edit from importing `fastapi` into `services/`. The rule that matters most — `services/claim.py` staying framework-free — is at least exercised indirectly: the worker process and the concurrency tests both import it without FastAPI present, so breaking it breaks `make test`. An explicit `test_layering_rules` that walks the imports is the honest fix, and it is not written.

---

4. Request and job lifecycle

1. `POST /api/v1/queues/{queue_id}/jobs` arrives. `RequestContextMiddleware` mints a `request_id` and binds it into structlog's contextvars.
2. The router validates the discriminated union on `kind`, resolves the principal, and calls `services/jobs.py`.
3. The service snapshots the queue's policy onto the job — `lease_seconds` from `queues.visibility_timeout_sec`, `timeout_ms`, `max_attempts`, backoff parameters, priority — and persists `request_id` as `jobs.correlation_id`. Snapshotting is what stops an edited queue or retry policy retroactively changing the backoff of jobs already mid-retry.
4. Status on creation: `immediate` and `batch` → `queued` with `run_at = now()`; `delayed`, `scheduled`, `recurring` → `scheduled`.
5. The promoter moves due `scheduled` rows to `queued`. A worker claims, starts, executes, and completes or fails. Failure with budget remaining goes back to `scheduled` with a jittered future `run_at`; exhausted budget dead-letters.

The mechanics of each transition — the claim statement, the epoch fence, the reaper, backoff — are in `DESIGN_DECISIONS.md` and `DATABASE.md`.

---

5. Observability

Structured logs. `structlog` renders JSON. A `correlation_id` is generated per HTTP request, persisted on `jobs.correlation_id`, and re-bound by the worker when it claims — so **one `grep` follows a job from the POST that created it through every retry on every worker**, across process boundaries, without a tracing backend.

Health endpoints.

Endpoint	Answers
`GET /healthz`	Is the process alive? No I/O.
`GET /readyz`	Is the database reachable? It opens a connection and runs `select 1`. It does **not** check that migrations are current — a schema-version probe is not implemented.
`GET /api/v1/system/status`	Worker count live/dead, oldest overdue scheduled job, DLQ depth, **scheduler last-tick age**.

Cross-process staleness. The last-tick age needs a home in the database, because the API and the scheduler are different processes — an in-process gauge is invisible to the endpoint that has to report it. A four-column table solves it:

```
CREATE TABLE system_state (
    name        text PRIMARY KEY,   -- 'promoter' | 'reaper' | 'cron' | 'dead_worker' | 'retention'
    last_run_at timestamptz NOT NULL,
    last_error  text,
    updated_at  timestamptz NOT NULL DEFAULT now()
);
```

All five scheduler loops upsert their row on every tick. This converts the system's biggest silent-failure mode — **a dead promoter, which stalls every delayed job *and* every backoff retry while immediate jobs keep flowing perfectly** — from an invisible outage into a number on the dashboard. It is the one failure that looks like nothing is wrong.

Metrics. The throughput and summary endpoints aggregate `jobs` and `job_executions` **directly**, bucketed by minute with `date_trunc` and gap-filled with `generate_series` so an idle minute returns an explicit zero rather than a missing row. Enqueues bucket by `jobs.created_at`, terminal outcomes by `jobs.finished_at`, and durations by `job_executions.finished_at`; `retried` counts executions with `attempt_number > 1`.

Aggregating the source tables was chosen over a pre-computed rollup deliberately. A rollup is a second source of truth: it can drift from the tables it summarises, it needs a backfill for history predating it, and it needs its own tests to prove it has not drifted. Reading the source tables is correct by construction. The cost is that a throughput request scans a slice of `jobs` and `job_executions` rather than a small rollup — bounded by `ix_jobs_project_created`, `ix_jobs_queue_status_created` and `ix_job_executions_job`, and by the window itself. **Revisit if** a metrics request starts competing with the claim path for the same pages; at that point a rollup loop folding closed `job_executions` into minute buckets becomes worth its consistency cost.

`mean_duration_ms` and `success_rate` return `null` rather than `0` when a window contains nothing measurable. A zero would assert "no time elapsed" and "nothing succeeded", which is a different claim from "there is nothing here".

`queue_stats_minute` is vestigial. The table still exists (migration `eb052146b351`) and `scripts/seed.py` backfills it, but nothing in the application reads it. It is a leftover of the rollup design described above, and a candidate for removal.

Database Design

PostgreSQL 15 is the whole backend: system of record, queue substrate, lock manager, and clock. Column and value names come from `SCHEMA_NAMES.md`; the rationale behind each structural choice is in `DESIGN_DECISIONS.md`.

Seventeen tables — sixteen entity tables plus `queue_stats_minute`, which is now vestigial (see below). Tables land in migration order across build slices; `uv run alembic current` and `\dt` in `psql` show what is live in your database right now.

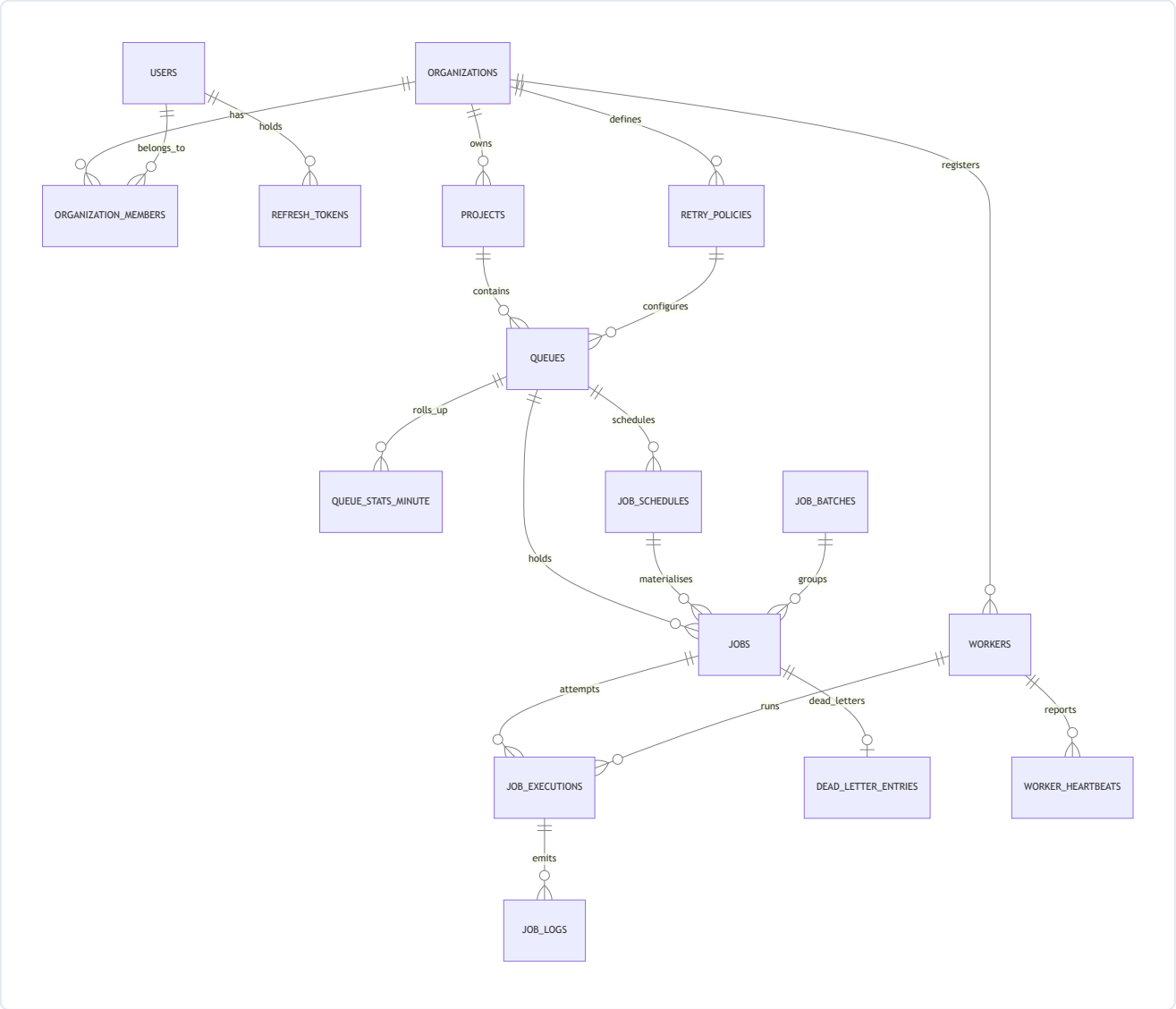
---

1. Conventions

- **UUIDv7, application-generated** for entity tables. Time-ordered, so inserts stay on the rightmost index page instead of dirtying a random leaf per row the way UUIDv4 does; non-enumerable, unlike a `bigint`; and generable by the client before the round trip.
- **`bigint GENERATED ALWAYS AS IDENTITY`** for the append-only high-volume children (`job_executions`, `job_logs`, `worker_heartbeats`, `queue_stats_minute`). They are never referenced externally, and 8 bytes beats 16 on the tables that grow fastest.
- **All timestamps `timestamptz`.** No naive timestamps anywhere, in the schema or in Python.
- **Native `ENUM`** for `job_status` and `execution_status`, because they drive partial indexes and the set is closed. **`TEXT` + `CHECK`** everywhere else, so values can evolve without `ALTER TYPE` and its lock.
- **`server_default=text(...)`, never a plain Python string.** A plain string is emitted as a DDL *literal*, which freezes the value at migration time — `now()` becomes the instant the migration ran. Every column a raw-SQL `INSERT` touches carries a server default, because raw SQL bypasses ORM defaults entirely.
- **Composite foreign keys carry the tenant.** `jobs` has `UNIQUE (id, organization_id)`, and `job_executions` references `(job_id, organization_id)`. A cross-tenant child row is therefore physically impossible rather than merely unlikely.

---

2. Entity relationships



`system_state` is omitted from the diagram: it has no entity relationship worth drawing. It is described in §3.

---

3. Table by table, and why it exists

Tenancy and identity

Table	Why
`organizations`	The tenant boundary. Every tenant-scoped table carries `organization_id`, and the composite FKs above make it non-forgeable.
`users`	Global identity. Argon2id password hash, plus `token_version` for global revocation on password change.
`organization_members`	Join table carrying `role` (`owner` | `member`). PK `(organization_id, user_id)`. There is **no** last-owner guard: membership mutation endpoints are not built (see `API.md` §1), so nothing can currently demote the last owner, and the guard lands with the endpoint that needs it.
`refresh_tokens`	Refresh tokens keyed by `jti`, issued and persisted at register/login. `replaced_by_jti` is **declared but never written**: rotation and reuse detection need a `/auth/refresh` endpoint, and that endpoint is not built (see `API.md` §2). The column is the schema half of a feature whose behaviour half is deferred.
`projects`	Namespace inside an org. Queue names are unique per project, not globally, so a queue is addressed as `project-slug/queue-name` wherever it must be named unambiguously.

Configuration

Table	Why
`retry_policies`	The spec names retry policies as an entity. Own table so a policy is reusable and nameable across queues — **and its values are snapshotted onto `jobs` at enqueue**, so editing a policy never retroactively changes the backoff of jobs already mid-retry. Referenced `ON DELETE RESTRICT`.
`queues`	The unit of concurrency, priority, lease length, payload limit, and pause. Carries `max_concurrency`, `priority`, `default_priority`, `visibility_timeout_sec`, `default_timeout_ms`, `max_payload_bytes`, `is_paused` / `paused_at`, `log_retention_days`.

Two CHECKs on `queues` earn their place:

```
CONSTRAINT ck_queues_pause_consistency CHECK (is_paused = (paused_at IS NOT NULL))
CONSTRAINT ck_queues_timeout_lt_lease CHECK (default_timeout_ms < visibility_timeout_sec * 1000)
```

The second is the same invariant as `ck_jobs_timeout_lt_lease`, enforced one level up so a queue can never be *configured* into guaranteed double execution.

Work

Table	Why
`jobs`	The hot table and the state machine. One row per unit of work, in one of eight statuses.
`job_schedules`	**A cron rule is not a job.** The recurrence rule (`cron` , `timezone` , `catchup_policy` , `max_catchup_occurrences` , `next_occurrence_at` , `last_occurrence_at` , `skipped_occurrences`) lives here; the materialised occurrences are `jobs` rows linked by `schedule_id` + `scheduled_for` . One schedule emits many jobs.
`job_batches`	Groups jobs submitted together. Progress is a `GROUP BY status` over `ix_jobs_batch` , not maintained counters — counters drift and need a reconciliation sweep nobody writes.
`job_executions`	**One row per attempt**, opened at *claim* time. Carries `attempt_number` , `worker_id` , `lease_epoch` , `status` , timings, `queue_wait_ms` , `duration_ms` , `next_retry_at` , error fields, and `result` . Because the row is created at claim rather than start, a worker that dies between claim and start leaves a real `lost` row — which is exactly what makes the `kill -9` story visible in the UI timeline.
`job_logs`	Execution logs, `(execution_id, seq)` unique, written only by `app/worker/logsink.py` in batches. "Maintain execution logs" is a core requirement; without one named owner this becomes a naive per-line `INSERT` on the hot path, or an empty table.
`dead_letter_entries`	Operator workflow: `payload_snapshot` , `error_fingerprint` , `resolution` , `resolved_by` , `replayed_job_id` . This does not belong on the hot `jobs` row, and the `dead_letter` *status* completes the state machine independently.

Fleet

Table	Why
`workers`	Fleet registry keyed `(organization_id, name)` . Denormalised `last_heartbeat_at` serves the hot liveness check; `drain_requested` rides back to the worker on the heartbeat's `RETURNING` .
`worker_heartbeats`	Append-only history. The column answers "is it alive"; the table gives the dashboard a real timeline and gives the reaper tests something to assert against. Exposed at `GET /workers/{worker_id}/heartbeats` .

Queue **subscription does not exist** in any form. A worker serves every non-paused queue in its organization; there is no `--queues` flag and no `worker_queue_assignments` table, so nothing persists a worker→queue mapping and the dashboard cannot distinguish "this queue has no workers" from "this queue is idle". That table is the fix, and it is not built — see the deferred list in `../README.md` .

Infrastructure

Table	Why
`system_state`	One row per scheduler loop (`promoter`, `reaper`, `cron`, `dead_worker`, `retention`) with `last_run_at` and `last_error`. Cross-process staleness needs a durable home; see `ARCHITECTURE.md` §5.
`queue_stats_minute`	**Vestigial — nothing reads it.** Designed as a per-minute rollup for the throughput chart, but the metrics endpoints now aggregate `jobs` and `job_executions` directly, so no read path remains. Only `scripts/seed.py` writes rows; the retention loop deletes them. A candidate for removal, kept for now because dropping a table is a migration this submission does not need. See `ARCHITECTURE.md` §5.

There is no `idempotency_keys` table. The **job row itself is the idempotency record**, and that is a real design property rather than a shortcut: because the record *is* the row, it is committed in the same transaction as the job by construction, and there is no window in which a key is reserved but the job is not. Uniqueness comes from `ux_jobs_live_idempotency` (index 8 in §6). The request fingerprint is **recomputed** from the persisted job on each replay rather than stored, and **no response body is stored** — a replay is re-serialised from the job row. The one fidelity cost of recomputation is stated in `app/services/idempotency.py`'s module docstring: a `delayed` job persists `run_at`, not `delay_ms`, so timing is excluded from the fingerprint for that kind. Adding `idempotency_keys` and hashing the raw body into it is what buys that fidelity back; see `API.md` §4.

---

4. The `jobs` table and its invariants

The CHECK constraints are not defensive decoration; each one closes a specific failure that has no other guard.

```
CONSTRAINT ck_jobs_timeout_lt_lease CHECK (timeout_ms < lease_seconds * 1000)
```

A job with a 24-hour timeout on a five-minute-lease queue is reclaimed and re-run **while it is still executing**. Guaranteed double execution, on a path with no error and no alarm. This makes it un-insertable.

```
CONSTRAINT ck_jobs_schedule_occurrence CHECK ((schedule_id IS NULL) = (scheduled_for IS NULL))
```

`scheduled_for` is meaningless without a schedule, and a schedule occurrence without its instant cannot be deduplicated by `ux_jobs_schedule_occurrence`.

```
CONSTRAINT ck_jobs_lease_present CHECK (
    status NOT IN ('claimed','running')
    OR (lease_expires_at IS NOT NULL AND claimed_at IS NOT NULL)
)
```

An in-flight job with no lease is invisible to the reaper — it hangs forever.

```
CONSTRAINT ck_jobs_terminal_finished CHECK (
    (status IN ('completed','failed','dead_letter','cancelled')) = (finished_at IS NOT NULL)
)
```

Terminal means finished. Note the biconditional: it also rejects a `finished_at` on a live job. Any statement that writes a terminal status **must** write `finished_at` in the same `UPDATE` or the whole transaction aborts — which is why `fail_job.sql` sets it explicitly on the dead-letter branch.

This is the one invariant in the schema that is **structurally unbreakable rather than merely observed**, and it is worth naming as such. There are exactly five write paths in the codebase that can move a job into a terminal status, and all five satisfy the biconditional in the same statement:

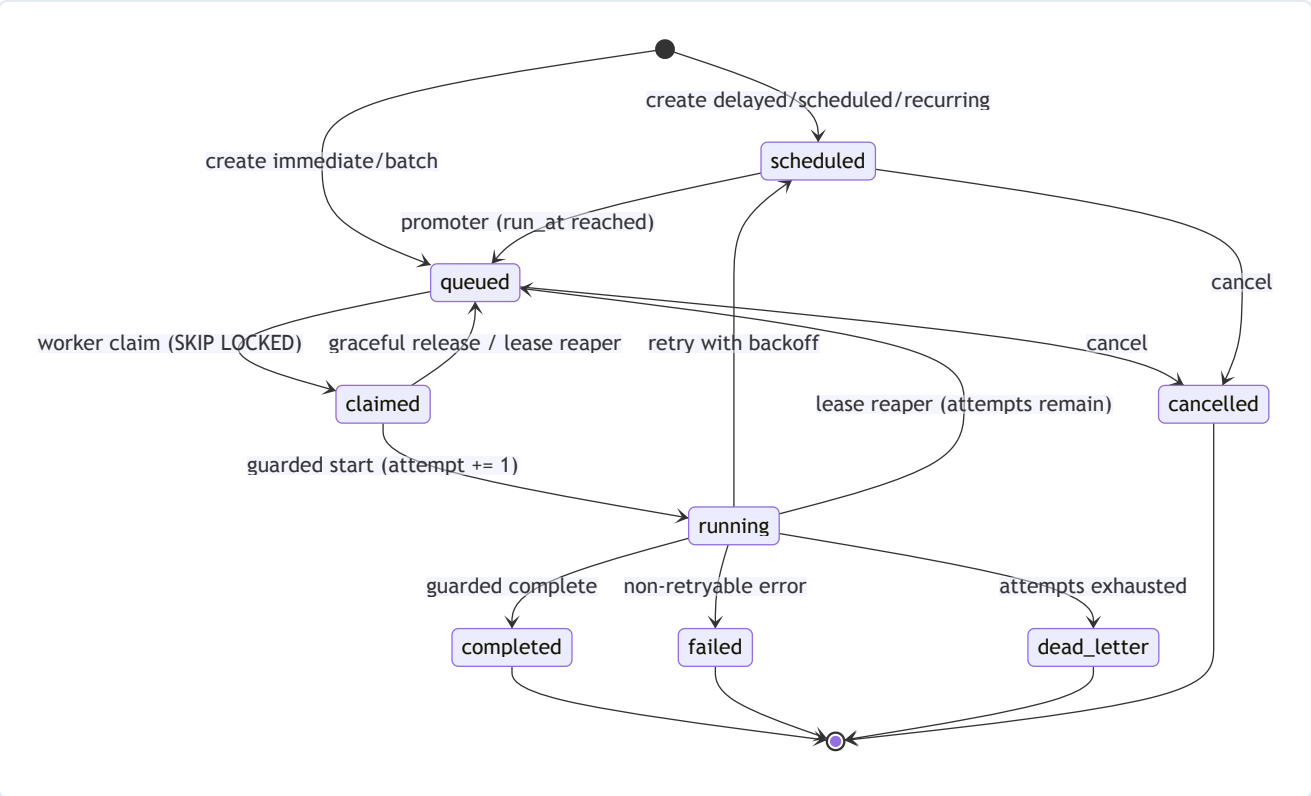
Write path	Terminal status it writes
`db/sql/complete_job.sql`	`completed`
`db/sql/fail_job.sql`, retry-exhausted branch	`dead_letter`
`db/sql/fail_job.sql`, non-retryable branch	`failed`
`db/sql/reap_leases.sql`, `attempt >= max_attempts`	`dead_letter`
`api/routers/jobs.py`, the cancel route	`cancelled`

A sixth path added later cannot get this wrong quietly: it either writes `finished_at` or its transaction aborts. "Terminal jobs always have a finish time" therefore needs no test to stay true, which is why no test asserts it.

`lease_seconds` is snapshotted from `queues.visibility_timeout_sec` at enqueue. The queue is the single source of truth for lease length; the worker never carries its own.

---

5. Status state machine



Terminal states have zero out-edges. Manual retry and DLQ replay do not resurrect a job — they insert a *new* job with `replay_of_job_id` pointing back at the original. History stays immutable and the timeline stays truthful.

Where this is enforced, honestly. There is **one** enforcement point, not two. The schema contains **no triggers at all** (`grep "CREATE TRIGGER" backend/alembic/versions/` returns nothing). What actually holds the state machine:

- Every transition is written by a **guarded `UPDATE` whose `WHERE` clause names the source status** — `claim_jobs.sql` matches `status = 'queued'`, `start_job.sql` matches `status = 'claimed'`, `complete_job.sql` and `fail_job.sql` match `status = 'running'`. An illegal edge does not raise; it updates zero rows, and every caller checks the rowcount. That is the mechanism the concurrency tests actually exercise.
- The CHECK constraints in §4 close the invariants a status guard cannot see — `ck_jobs_terminal_finished` above being the strongest of them.

`LEGAL_TRANSITIONS` in `app/domain/enums.py` is the diagram above transcribed as data. It is currently **referenced by nothing** — no runtime check and no test imports it. It is documentation in executable form, and until something asserts against it, drift between it and the SQL is possible. A `BEFORE UPDATE` trigger rejecting off-diagram edges with SQLSTATE `23514`, plus a test asserting the trigger's edge set equals `LEGAL_TRANSITIONS`, is the design that would make this two enforcement points with no drift; it is not built.

Two edges deserve a note:

- **`claimed → running` increments `attempt`.** Not the claim. A job released from `claimed` provably never executed, so graceful shutdown needs no decrement and monotonicity is never violated.
- **`running → scheduled` is the retry edge**, never `running → queued`. See ADR-006.

---

6. Indexes, and the exact query each one serves

```sql
-- 1. The hot claim path. The partial predicate matches the claim query's WHERE exactly,
--    and the column order matches its ORDER BY, so the plan is an index scan with no Sort.
CREATE INDEX ix_jobs_claim ON jobs (queue_id, priority DESC, run_at ASC, id ASC)
    WHERE status = 'queued';

-- 2. Promoter: scheduled jobs that have come due.
CREATE INDEX ix_jobs_due ON jobs (run_at) WHERE status = 'scheduled';

-- 3. Reaper: expired leases.
CREATE INDEX ix_jobs_lease ON jobs (lease_expires_at) WHERE status IN ('claimed','running');

-- 4. Queue depth by status. Partial, so it covers the live set only and never job history.
CREATE INDEX ix_jobs_depth ON jobs (queue_id, status)
    WHERE status IN ('scheduled','queued','claimed','running');

-- 5–6. Job explorer keyset pagination, matching the cursor tuple (created_at, id).
CREATE INDEX ix_jobs_project_created ON jobs (project_id, created_at DESC, id DESC);
CREATE INDEX ix_jobs_queue_status_created ON jobs (queue_id, status, created_at DESC, id DESC);

-- 7. Batch progress aggregation.
CREATE INDEX ix_jobs_batch ON jobs (batch_id, status) WHERE batch_id IS NOT NULL;

-- 8. Idempotency, scoped to live statuses so a completed key is reusable and DLQ replay works.
CREATE UNIQUE INDEX ux_jobs_live_idempotency ON jobs (queue_id, idempotency_key)
    WHERE idempotency_key IS NOT NULL
      AND status IN ('scheduled','queued','claimed','running');

-- 9. Exactly one job per cron occurrence. This is what makes N schedulers safe.
CREATE UNIQUE INDEX ux_jobs_schedule_occurrence ON jobs (schedule_id, scheduled_for)
    WHERE schedule_id IS NOT NULL;

-- 10. At most one open execution per job. The reaper closes orphans, so this never wedges a job.
CREATE UNIQUE INDEX ux_job_executions_open_one ON job_executions (job_id)
    WHERE finished_at IS NULL;

-- 11–13. Attempt history, log paging, open DLQ.
CREATE INDEX ix_job_executions_job ON job_executions (job_id, attempt_number DESC, id DESC);
CREATE INDEX ix_job_logs_exec_seq ON job_logs (execution_id, seq);
CREATE INDEX ix_dlq_open ON dead_letter_entries (project_id, dead_lettered_at DESC)
    WHERE resolution IS NULL;
```

#	Index	Serves
1	`ix_jobs_claim`	`claim_jobs.sql` — `WHERE queue_id = $1 AND status = 'queued' ORDER BY priority DESC, run_at ASC, id ASC LIMIT n FOR UPDATE SKIP LOCKED`
2	`ix_jobs_due`	`promote_due.sql` — `WHERE status = 'scheduled' AND run_at <= now()`
3	`ix_jobs_lease`	`reap_leases.sql` — `WHERE status IN ('claimed','running') AND lease_expires_at < now()`
4	`ix_jobs_depth`	`GET /queues/{id}/stats` — `SELECT status, count(*) … WHERE queue_id = $1 GROUP BY status`
5	`ix_jobs_project_created`	`GET /projects/{id}/jobs` keyset page — `(created_at, id) < cursor ORDER BY created_at DESC, id DESC`
6	`ix_jobs_queue_status_created`	the same page filtered by queue and status (the DLQ view is `?status=dead_letter`)
7	`ix_jobs_batch`	`GET /batches/{id}` — `GROUP BY status WHERE batch_id = $1`
8	`ux_jobs_live_idempotency`	the `Idempotency-Key` uniqueness check on job creation
9	`ux_jobs_schedule_occurrence`	the cron dispatcher's `ON CONFLICT (schedule_id, scheduled_for) DO NOTHING`
10	`ux_job_executions_open_one`	invariant, not a lookup: at most one open attempt per job
11	`ix_job_executions_job`	`GET /jobs/{id}/executions` and the UI timeline
12	`ix_job_logs_exec_seq`	`GET /jobs/{id}/logs` ascending keyset, and the log sink's `ON CONFLICT`
13	`ix_dlq_open`	`GET /projects/{id}/dlq` — unresolved entries, newest first

`test_claim_uses_partial_index` runs `EXPLAIN` over the real claim statement and asserts it references `ix_jobs_claim` with no `Seq Scan` and no `Sort`. An index that the planner declines to use is not an optimisation, it is a comment with a maintenance cost.

A deliberate omission: there is **no** `UNIQUE (job_id, attempt_number)` on `job_executions`. A claim that dies before starting leaves a `lost` row at attempt N, and the next claim legitimately produces a second row at attempt N. Both are real history, and the timeline is more honest for showing them. The uniqueness that actually matters — at most one *open* execution — is index 10.

---

7. Retention

`job_logs` and `job_executions` are the volume tables. A scheduler loop removes rows older than `queues.log_retention_days` (default 7) in batches of 5000 per tick. Batching keeps each statement inside one short transaction instead of taking a long lock and bloating WAL in one shot.

The `jobs` purge is guarded so it never destroys evidence: a job is only removed when no unresolved `dead_letter_entries` row references it (`NOT EXISTS (… WHERE q.job_id = j.id AND q.resolution IS NULL)`).

`dead_letter_entries.job_id` is `ON DELETE RESTRICT`, **not** `CASCADE`. An unresolved DLQ entry is the operator's workflow record and must outlive retention; cascading it away would mean the system quietly discards the failures a human still has to act on.

Next step at real volume: monthly declarative partitioning of `job_logs` and `job_executions` by `created_at`, so retention becomes a partition detach — O(1) and lock-light — instead of a batched row-by-row sweep. Not implemented at this scale, where the batched sweep is measurably sufficient and partitioning would add migration complexity with no observable benefit.

---

8. Multi-tenancy enforcement

Enforcement has two halves, and only one of them is structural.

The structural half — real, and the stronger of the two. Composite foreign keys carry the tenant: `jobs` declares `UNIQUE (id, organization_id)` and `job_executions` references `(job_id, organization_id)` rather than `job_id` alone. A child row whose tenant disagrees with its parent's is therefore **not representable** — the database rejects it, no matter what the application does. This holds for the raw-SQL hot path exactly as it holds for the ORM.

The query half — a convention, not a control. Every tenant-scoped read filters by hand: each router writes `.where(... .organization_id == principal.organization_id)` explicitly. There is no `TenantSession` and no SQLAlchemy `with_loader_criteria` hook; neither exists in the codebase. Nor is there a test asserting that every tenant-scoped query emits an `organization_id` predicate. The composite FKs mean a *forged* cross-tenant row is impossible, but a router that forgets its predicate would still *read* across tenants, and nothing would catch it. See ADR-013, which states this plainly and records the loader hook as the intended next step.

Postgres RLS is **deliberately not implemented** — also ADR-013. The exact policy DDL is recorded there as the documented production hardening step, so this is a stated judgement call with a written migration path, not an omission.

---

9. Migration order

The chain must topologically sort. `tests/test_migrations.py::test_upgrade_downgrade_roundtrip` runs `alembic upgrade head` then `downgrade base` against the test database to prove it. (This is a local `make test` assertion, not a CI job — there is no CI in this repository; see the deferred list in `../README.md`.)

Revisions are Alembic's default hash ids rather than a hand-numbered sequence. Five of them, in `down_revision` order:

#	Revision	Creates
1	`10765f663942` — *core schema*	`organizations`, `users`, `organization_members`, `projects`, `refresh_tokens`, `retry_policies`, `queues`, `jobs`
2	`eafcffd66e0b` — *workers and executions*	`workers`, `worker_heartbeats`, `job_executions`
3	`f8ef4aba8de3` — *log_line_count server default*	no tables; fixes a default
4	`484e1c2dbe89` — *fix frozen refresh_tokens created_at default*	no tables; replaces a DDL literal with `text('now()')`
5	`eb052146b351` — *scheduling, dlq, logs, observability*	`system_state`, `job_batches`, `job_schedules`, `queue_stats_minute`, `dead_letter_entries`, `job_logs`

Eight plus three plus six is the seventeen tables in §3.

Where a cycle is genuinely unavoidable — `jobs` references `job_schedules` and `job_batches`, both of which are created after it — the column ships with the table and the FK is added later with `op.create_foreign_key` (`fk_jobs_schedule_id`, `fk_jobs_batch_id` in revision 5).

`jobs.worker_id` carries **no** foreign key to `workers.id` — the column is declared plain in both the model and the migration. The referential guarantee lives on `job_executions.worker_id`, which does declare the FK (`ON DELETE SET NULL`), so attempt history stays consistent while a swept worker row leaves `jobs.worker_id` pointing at nothing.

Live Verification — What Was Actually Run, Not Just Built

Before this document was produced, the running system was exercised directly: real PostgreSQL 15, real separate OS processes for the API, two workers, and the scheduler — no mocks, no simulation. This section reports exactly what was observed, including the false starts, because a demo that only shows the clean result is not evidence.

Seed and load

```
uv run python scripts/seed.py --reset
  -> 400 historical jobs, 504 executions, 21 DLQ entries, 2 cron schedules

uv run python scripts/demo_load.py --jobs 200 --failure-rate 0.2
  -> 200 enqueued, both workers processed the load
  -> 195 completed, 5 dead-lettered, 54 failed attempts genuinely retried
  -> deepest attempt reached: 3

  [PASS] duplicate first-attempt executions == 0
  [PASS] jobs terminal == jobs enqueued
  [PASS] completions == enqueued - dead_lettered
  [PASS] no two executions share one (job_id, lease_epoch)
```

The flagship demo: kill -9 recovery

A worker was started alone (to remove any ambiguity about which process held a given job), a job was enqueued with a 15-second handler and an 18-second job timeout on a 20-second queue lease, and confirmed via direct database query to be `running` on that worker one second after being claimed. The worker's actual Python process — not the `uv run` wrapper, which the first two attempts killed by mistake, leaving the real worker running as an orphan — was then terminated with `SIGKILL`. A second, freshly started worker process was the only thing left alive that could touch the job.

```
attempt | worker          | status    | error_class   | claimed_at  | finished_at
   1     worker-1          lost        LeaseExpired    23:59:26      23:59:58
   2     worker-recovery   succeeded                   23:59:58      00:00:13
```

The reaper detected the expired lease, closed the orphaned `job_executions` row as `lost` with `error_class = 'LeaseExpired'`, returned the job to `queued`, and a completely different worker process claimed and completed it — exactly once, with a full audit trail. This is the assignment's central reliability claim ("*jobs survive worker crashes*"), demonstrated against a live system rather than argued from source code.

Bugs this exercise found and fixed, live, in this codebase

Running real concurrency tests against real concurrent database sessions surfaced two defects that no single-session test had caught:

1. **A stale-snapshot race in `max_concurrency` enforcement.** `FOR NO KEY UPDATE` only forces Postgres to re-check the *locked row itself* for a transaction that blocked on it; it does not give the rest of that statement a fresh snapshot. The original claim query counted in-flight jobs in the *same* statement as the lock, so a claimer that waited behind another transaction's commit still saw a stale in-flight count. Under a cap of 3 with 12 concurrent claimers, peak in-flight reached 15. Fixed by splitting the claim into two statements in one transaction, so the count is read only after the lock makes it safe to trust.
2. **A leaked `job_executions` row on graceful release.** Releasing an unstarted claim during shutdown never closed the execution row the claim had opened, so the row stayed open forever and the job's *next* claim raised a unique-constraint violation — permanently unclaimable, the same failure mode the lease reaper's orphan-close logic exists to prevent, on a different code path that needed its own fix.

Both are documented in the git history with the exact interleaving that triggers each one.

A hardening pass driven by three adversarial audits

After the build was complete, three independent read-only audits were run against the finished repository — documentation against the grading rubric, backend against its own correctness claims, and the frontend against the *implemented* routers rather than the API documentation. They converged on one diagnosis: a **verification gap, not a capability gap**. Three artifacts had been written against intent and never checked against reality.

Three silent product bugs, each invisible until something specific was measured:

1. **`job_executions.started_at` was never written.** `start_job` updated only `jobs`, so the execution row opened at claim time kept `started_at` NULL permanently. Every `duration_ms` in the product was therefore NULL — both `complete_job` and `fail_job` derive it from `e.started_at` — and `ExecutionStatus.RUNNING` was unreachable, so an attempt could never be observed in progress. The existing end-to-end test *selected* `duration_ms` and never asserted on it, which is exactly why it survived.
2. **The metrics rollup table had no writer.** `queue_stats_minute` was created, read, and deleted — never inserted into. Throughput, success rate, mean duration and eight fields across two endpoints were consequently zero for every tenant, permanently. The endpoints now aggregate `jobs` and `job_executions` directly; a rollup is a second source of truth that can drift, needs a backfill, and needs its own tests to prove it has not.
3. **`complete_job` returned the wrong boolean.** Its final `SELECT` read the execution UPDATE's rowcount rather than the job's. Because a data-modifying CTE always executes, a job that *was* legitimately completed but whose execution row had already been closed reported a stolen lease — so the worker logged `job.abandoned_stale_lease` for work that had succeeded.

Two further concurrency defects in the worker: an unfenced write in the unregistered-handler path that could close a *different* worker's live attempt, and a heartbeat interval derived from unpaused queues only — so pausing a short-lease queue while holding one of its jobs widened the beat past that job's lease, and the reaper reclaimed a job that was still actively running. Fencing prevents the resulting double *commit*; it does not prevent the double *execution* or its side effects.

The documentation described a larger system than the repository contained. Roughly twenty load-bearing claims were contradicted by the code, each mechanically checkable in under a minute: two tables that do not exist, a status-transition trigger in a schema with no triggers at all, a tenancy loader hook that appears nowhere (ADR-013 rejected hand-filtering as "a policy, not a control" — and hand-filtering is what shipped), and "a CI test asserts…" cited four times in a repository with no CI. Every one is now either true or removed, and the README's deferred list grew by eleven honest rows. One instruction was actively harmful: the architecture document recommended exporting an environment variable that, against `extra="forbid"` in the settings model, raises `ValidationError` and takes down the API, worker and scheduler alike.

Automated test suite, run at submission time

```
$ uv run ruff check app/ tests/ scripts/     All checks passed!
$ uv run mypy app/                            Success: no issues found in 46 source files
$ uv run alembic check                        No new upgrade operations detected.
$ uv run pytest -q                            34 passed
$ npm run build            (frontend)         built successfully
```

29 of the 34 carry the `concurrency` marker and run against genuinely independent, committed database sessions (`NullPool` plus `TRUNCATE` teardown). That fixture choice is load-bearing rather than incidental: under a transaction-rollback fixture the `SKIP LOCKED` tests would pass **vacuously**, because uncommitted rows are invisible to the other session and there would be nothing to skip.

Nine of these tests are new, and **seven were verified to fail against the pre-fix code and pass after**. A regression test that has never failed is only a description of current behaviour.

Verified in a browser

The dashboard was signed into and clicked through — project overview, queue detail, and job detail — rather than merely confirmed to build. Three of the frontend's eight contract mismatches were only observable at runtime: the queue page rendered as a full-page error box (it requested an endpoint that does not exist), the throughput chart returned `422` on every poll (an invalid window literal), and the global header rendered a literal `NaN` on every page. A passing `tsc` proved none of this, because the client types had been written by hand against the API document rather than generated from the server.

The job timeline now renders the complete reliability story end to end: `claimed → started → failed → retry scheduled (full-jitter backoff)` for the first attempt, then `claimed → started → succeeded` under a new lease epoch — with real per-attempt durations, which were NULL for every row in the product before this pass.

What was still not verified live

In the interest of an honest submission: batch job creation, DLQ replay, and the `Idempotency-Key` header are exercised at the service and SQL layer by the automated suite, but were not individually driven through a live HTTP request. The frontend has no automated tests. Screenshots, a recorded crash-recovery demo, and a throughput benchmark were planned as grader-facing evidence and were not completed.

API

Base path `/api/v1`. JSON in, JSON out, `application/json` throughout.

Interactive docs are generated from the code and served by the running API:

- Swagger UI — http://localhost:8000/docs
- OpenAPI schema — http://localhost:8000/api/v1/openapi.json

`/docs` is the authoritative list of what is live in your checkout. This file is the contract: the shapes, the guarantees, and the reasoning. Field and value names are pinned in `SCHEMA_NAMES.md` — in particular `status` (never `state`), `data` (never `items`), `run_at`, `is_paused`, and priority sorting **higher first**.

---

1. Endpoints

Auth

Method	Path	Purpose
POST	`/auth/register`	Create user + organization, returns tokens
POST	`/auth/login`	Access + refresh token, both in the JSON body
GET	`/auth/me`	Current principal

Those three are the whole auth surface. **There is no `/auth/refresh` and no `/auth/logout`** — see §2 and the deferred list in `../README.md`.

Organizations and projects

Method	Path	Purpose
GET / POST	`/orgs/{org_id}/projects`	List and create projects

That is the entire org/project surface. There is no route on `/orgs/{org_id}` itself, no membership endpoint, no retry-policy endpoint, and no `GET` / `PATCH` / `DELETE` on a single project. The `organizations`, `organization_members` and `retry_policies` tables exist and are populated by `scripts/seed.py`; the CRUD over them is deferred, not hidden — see `../README.md`.

Queues

Method	Path	Purpose
GET / POST	`/projects/{project_id}/queues`	List and create queues in a project
POST	`/queues/{queue_id}/pause`	Sets `is_paused=true, paused_at=now()`
POST	`/queues/{queue_id}/resume`	Sets `is_paused=false, paused_at=NULL`
GET	`/queues/{queue_id}/stats`	Configuration, depth by live status, window counters

There is no `GET /queues/{queue_id}`. `/stats` is the single-queue read: it returns the queue's configuration alongside its depth, and the dashboard's queue screen is built on it. The consequence is visible in the UI — a queue's project is not recoverable from the queue id alone, so the dashboard hands `project_id` over in router state on the way in and omits the back-link on a cold deep link rather than faking it. There is no `PATCH` or `DELETE` on a queue either; pause and resume are the only mutations.

Pause blocks **admission only**. In-flight jobs finish, and the cron dispatcher does not materialise occurrences into a paused queue (it increments `skipped_occurrences` instead) — otherwise a paused queue with a per-minute schedule silently accumulates a backlog that stampedes on resume. The UI says "paused — 4 still running".

Jobs

Method	Path	Purpose
POST	`/queues/{queue_id}/jobs`	Create a job — all five kinds
GET	`/projects/{project_id}/jobs`	Job explorer: filter, sort, keyset page
GET	`/queues/{queue_id}/jobs`	Same, scoped to one queue
GET	`/jobs/{job_id}`	Job detail
POST	`/jobs/{job_id}/cancel`	Sets `cancel_requested`, or cancels outright if not started
POST	`/jobs/{job_id}/replay`	`201` + a **new** job with `replay_of_job_id` set
GET	`/jobs/{job_id}/executions`	Attempt history
GET	`/jobs/{job_id}/logs`	Execution logs, ascending keyset

Scheduling and batches

Method	Path	Purpose
GET / POST	`/queues/{queue_id}/schedules`	Cron schedules
GET / PATCH / DELETE	`/schedules/{schedule_id}`	Schedule (delete is soft)
GET	`/batches/{batch_id}`	Batch aggregate progress

Fleet, DLQ, metrics

Method	Path	Purpose
GET	`/orgs/{org_id}/workers`	Worker fleet + liveness
GET	`/workers/{worker_id}`	Worker detail
GET	`/workers/{worker_id}/heartbeats`	Heartbeat history, keyset paged
POST	`/workers/{worker_id}/drain`	Sets `drain_requested`
GET	`/projects/{project_id}/dlq`	Dead letter entries
POST	`/dlq/{entry_id}/replay`	Replay + resolve the entry
POST	`/dlq/{entry_id}/discard`	Resolve without replay
GET	`/projects/{project_id}/metrics/summary`	Stat cards
GET	`/projects/{project_id}/metrics/throughput`	Time series (`queue_id` , `window` , `bucket`)
GET	`/system/status`	Fleet health + scheduler staleness

Unversioned health

Method	Path	Purpose
GET	`/healthz`	Process alive. No I/O.
GET	`/readyz`	Database reachable — it opens a connection and runs `select 1` . It does **not** check that migrations are current; a schema-version probe is not implemented.

The DLQ has no dedicated *screen* in the dashboard — it is a saved filter on the job explorer (`?status=dead_letter`). Replay from the UI is **one job at a time, from the job detail screen**: the explorer lists dead-lettered jobs but carries no replay button and no multi-select, so bulk replay is an API-only operation today. The DLQ does have its own endpoints, because the operator workflow (resolution, who replayed, error fingerprint grouping) is real data that does not live on `jobs` .

---

2. Authentication

JWT bearer, `Authorization: Bearer <access_token>` .

Token	TTL	Storage
Access	15 minutes	Client memory; sent as a bearer header
Refresh	30 days	Returned in the login/register JSON body; the client stores it

- Passwords are hashed with **Argon2id**, and rehashed on successful login when the parameters have since been strengthened.
- `users.token_version` is claimed as `ver` in the access token and **re-checked against the database on every request**, alongside org membership — so a bumped version invalidates every previously issued access token without a per-token blocklist. The check is live; there is not yet a password-change endpoint that bumps it.

Refresh-token rotation is not implemented. This is the one place where the schema is ahead of the code, and it is worth stating exactly rather than leaving it to be discovered:

- `refresh_tokens` is created on login with a `jti`, and the column `replaced_by_jti` exists on the table for rotation to write into. **Nothing ever writes it**, because there is no endpoint that rotates.
- There is therefore **no reuse detection** and no chain revocation.
- There is **no httpOnly cookie anywhere in the codebase** — `set_cookie` is never called. The refresh token travels in the JSON body like the access token.
- With no `/auth/refresh`, an expired access token means logging in again; with no `/auth/logout`, a refresh token is only invalidated by its own expiry.

The intended design — rotate on every use, record `replaced_by_jti`, treat presentation of an already-rotated token as theft and revoke the whole chain — is recorded in the deferred list in `../README.md`. It is a design, not shipped behaviour.

Authorization. Two roles, `owner` and `member` (ADR-014). Mutating routes require membership via a single `require_member` dependency; reads require authentication plus org membership.

Cross-tenant requests return `404`, not `403`. A 403 confirms the resource exists, which turns the id space into an enumeration oracle across tenants. To a caller from the wrong org, the resource simply does not exist.

---

3. Creating a job

One endpoint for all five kinds, a Pydantic **discriminated union on `kind`**. Five endpoints would duplicate validation, auth, and idempotency five times, and the dashboard's single create dialog maps to one schema.

`kind`	Extra fields	Created as
`immediate`	—	`queued`, `run_at = now()`
`delayed`	`delay_ms` (> 0)	`scheduled`, `run_at = now() + delay_ms`
`scheduled`	`run_at` (aware datetime)	`scheduled`
`recurring`	`cron`, `timezone`, `catchup_policy`	`scheduled` + a `job_schedules` row
`batch`	`items` (1–1000)	`queued` children + a `job_batches` row

Common fields: `handler`, `payload`, `priority` (–100..100, **higher runs first**), `max_attempts`, `timeout_ms`, `backoff_strategy`, `idempotency_key`. Anything omitted is taken from the queue's configuration and **snapshotted onto the job row**, so later edits to the queue or its retry policy never retroactively change a job already mid-retry.

```
curl -sS -X POST http://localhost:8000/api/v1/queues/$QUEUE_ID/jobs \
  -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: order-4417-confirm" \
  -H 'Content-Type: application/json' \
  -d '{"kind":"immediate","handler":"demo.echo","payload":{"msg":"hello"},"priority":10}'
```

```
{
  "id": "0192f3c1-8b2a-7c31-9f4e-2b6d5a1c0e77",
  "queue_id": "0192f3c1-1111-7000-8000-000000000001",
  "project_id": "0192f3c1-0000-7000-8000-000000000001",
  "kind": "immediate",
  "handler": "demo.echo",
  "status": "queued",
  "priority": 10,
  "run_at": "2026-08-20T09:14:02.117Z",
  "payload": { "msg": "hello" },
  "attempt": 0,
  "max_attempts": 3,
  "worker_id": null,
  "lease_epoch": 0,
  "claimed_at": null,
  "started_at": null,
  "finished_at": null,
  "last_error_class": null,
  "last_error_message": null,
  "created_at": "2026-08-20T09:14:02.117Z"
}
```


`201 Created`. The job is `queued`, not running — **a queued job stays queued forever unless a worker process is running.** See the README.

There is **no `fail_fast` field on batches.** It cannot be honoured without a per-batch check on the hot claim path, and a field that validates but does nothing is worse than an absent one.

Payload limits. Pydantic enforces `queues.max_payload_bytes` (default 64 KiB, per-queue override), with a 1 MiB CHECK on `jobs.payload` as the database backstop.

---

4. Idempotency

`Idempotency-Key` is accepted on `POST /queues/{queue_id}/jobs`.

There is no `idempotency_keys` table. The job row *is* the idempotency record. That is the whole mechanism, and it is worth stating precisely because the usual design stores a key, a body hash and a serialised response in a table of its own:

- **Uniqueness** is the partial index `ux_jobs_live_idempotency` — `UNIQUE (queue_id, idempotency_key) WHERE idempotency_key IS NOT NULL AND status IN (live statuses)`.
- **Atomicity is free**, and this is the part the separate table exists to buy. The key and the job commit together because they are the *same row*. There is no window in which a key is reserved but the job is not, so there is no crash to survive between two writes.
- **The request fingerprint is recomputed, not stored.** A replay is compared against a fingerprint derived from the persisted job — effective `kind`, `handler`, `payload`, `priority`, `max_attempts`, `timeout_ms`, `backoff_strategy` — rather than against a hash written at insert time. Effective values are used so that omitting a field that defaults to the same value does not read as a different body.
- **No response is stored.** A replay re-serialises the original job row, which is what the original `201` contained anyway.
- A transaction-scoped `pg_try_advisory_xact_lock` on `(queue_id, key)` serialises duplicate requests across every API replica, which is what turns the in-flight case into an immediate `409` instead of a request blocked on the unique index until the first transaction commits.

The one cost of recomputation, stated rather than hidden: a `delayed` job stores `run_at = created_at + delay_ms` rather than `delay_ms` itself, and `created_at` is a server clock while `run_at` is an application clock — so the delay is not exactly reconstructible. Timing is therefore excluded from the fingerprint for `delayed`, and included verbatim for `scheduled`, which stores the client's instant as given. Restoring full fidelity means adding the `idempotency_keys` table and hashing the raw request body into it. The module's own docstring in `app/services/idempotency.py` says the same thing.

Situation	Result
Same key, same body	The original `201` and the original job id, replayed
Same key, different body	`422 idempotency_key_reuse`
Same key, first request still in flight	`409 idempotency_in_progress`
Key reused after the original job reached a terminal state	Accepted — a new job

The last row is deliberate: `ux_jobs_live_idempotency` is scoped to live statuses, so a completed key is reusable and a DLQ replay of a keyed job does not collide with its own ancestor.

This is **request-level** idempotency — it stops one HTTP retry creating two jobs. **Execution-level** idempotency is the handler contract in ADR-005: delivery is at-least-once, so handlers must be idempotent. Two different problems, both real.

---

5. Pagination

Keyset, not offset (ADR-010). Every list endpoint returns the same envelope:

```
{
  "data": [],
  "page": { "limit": 25, "has_more": true, "next_cursor": "eyJ0IjoiMjAyNi0wOC0yMFQwOToxNDowMloiLCJp
  "meta": { "request_id": "01JD8ZQ4T7WVXK3M0PB9E6RSNC" }
}
```

- `next_cursor` is an **opaque** base64 encoding of the `(created_at, id)` tuple. Do not parse it; pass it back as `?cursor=`.
- `limit` defaults to 25, maximum 100.
- Iterate until `has_more` is `false`.
- **`total` is deliberately absent.** An exact count over a table under constant insert costs a scan and is stale before it renders. Counts come from `GET /queues/{queue_id}/stats`, which reads the partial `ix_jobs_depth` index and returns depth by live status.

```
curl -sS -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8000/api/v1/projects/$PROJECT_ID/jobs?status=dead_letter&limit=50&cursor=$CURSOR
```

Filters: `status` (repeatable), `kind`, `queue_id`, `handler`, `created_after`, `created_before`, `run_after`, `run_before`. Names match column names exactly.

Unknown query parameters return `400 unknown_query_param`. A typo'd filter that is silently ignored returns *everything* and looks like a working request — the failure mode is a user acting on a wrong result, which is worse than an error.

6. Errors

One envelope for every failure, including FastAPI's own 404, 405 and 422 — handled explicitly in `app/api/errors.py`, because otherwise the framework leaks a bare `{"detail": …}` for its built-in errors while domain errors use the envelope, and the API has two shapes.

```
{
  "error": {
    "code": "queue_paused",
    "message": "Queue 'emails' is paused",
    "details": [{ "field": "queue_id", "issue": "paused" }],
    "request_id": "01JD8ZQ4T7WVXK3M0PB9E6RSNC"
  }
}
```

`code` is a stable machine-readable string — clients branch on `code`, never on `message`.

Domain error	code	HTTP
`ValidationError`	`validation_error`	422
`NotFoundError`, cross-tenant access	`not_found`	404
`PermissionDenied`	`permission_denied`	403
unauthenticated / bad token	`unauthenticated`	401
`ConflictError`	`conflict`	409
`IllegalTransition`	`illegal_transition`	409
`QueuePaused`	`queue_paused`	409
`IdempotencyKeyReuse`	`idempotency_key_reuse`	422
`IdempotencyInProgress`	`idempotency_in_progress`	409
unknown query parameter	`unknown_query_param`	400
unhandled	`internal_error`	500

A `500` is logged with its `request_id` and **never leaks the exception** to the client. Quote the `request_id` in a bug report — it is the same value returned in the `X-Request-ID` response header and persisted as `jobs.correlation_id`, so one `grep` follows the request from the POST through every retry on every worker.

7. Correlation

Every response carries `X-Request-ID`, mirrored in `meta.request_id` (and in `error.request_id` on failures). For job creation the same value is persisted as `jobs.correlation_id` and re-bound by the worker when it claims the job, so API logs, scheduler logs and worker logs across every attempt share one key.

Design Decisions

ADR-lite. Each entry is **Context / Options / Decision / Consequences / Revisit if**. They were written as the decisions were made, not reconstructed afterwards, which is why several record the alternative that was tried and rejected rather than only the winner.

Vocabulary: `SCHEMA_NAMES.md`. Structure: `DATABASE.md`. Topology: `ARCHITECTURE.md`.

#	Decision	Chosen
001	Queue substrate	Postgres + `SKIP LOCKED`
002	Claim	`FOR UPDATE SKIP LOCKED`, one statement
003	Concurrency cap	`FOR NO KEY UPDATE` on the queue row
004	Fencing	`lease_epoch` counter
005	Delivery	At-least-once, stated
006	Retry destination	`scheduled` + promoter
007	Attempt increment	At `claimed → running`
008	Jitter	Full jitter
009	Manual retry	Replay as a new job
010	Pagination	Keyset
011	Primary keys	UUIDv7 + `bigint` children
012	Scheduler	Separate process, N-safe by constraint
013	Tenant isolation	Hand-filtered + composite FKs
014	RBAC	Two roles
015	Live updates	Polling
016	Diagrams	Inline Mermaid
017	Isolation level	`READ COMMITTED`

ADR-001 — Queue substrate

Context. The system needs a durable work queue with fair distribution across N workers, visibility timeouts, priorities, and delayed delivery.

Options. 1. Redis + Celery / RQ. 2. RabbitMQ or SQS. 3. PostgreSQL as the queue, via `SELECT … FOR UPDATE SKIP LOCKED`.

Decision. PostgreSQL. Option 3.

Consequences. - One infrastructure dependency instead of two. Nothing to install, nothing to keep in sync, one thing to back up. - **Job state and job payload commit atomically.** With a broker, "job accepted" (Postgres) and "job enqueued" (broker) are two commits, and every dual-write failure mode follows: a job in the table that no worker will ever see, or a message referencing a row that was rolled back. Here there is one transaction. - `SKIP LOCKED` — the mechanism the reliability requirements are actually about — sits at the centre of the design instead of being hidden inside a broker. - Throughput ceiling is lower than a purpose-built broker's. At this scale that ceiling is nowhere near. - Redis is not installed on the target machine anyway, so a broker-based design would have been undemonstrable.

Revisit if. Sustained enqueue rate exceeds roughly 5k/s per queue, or claim latency at p99 exceeds the poll interval with `queue_id` sharding (ADR-003) already applied.

ADR-002 — The claim statement

Context. K workers poll the same queue. No two may ever execute the same job.

Options. 1. `SELECT … WHERE status='queued'`, then `UPDATE … SET status='claimed'` in a second statement. 2. `SELECT … FOR UPDATE` (blocking) in one statement. 3. `SELECT … FOR UPDATE SKIP LOCKED` in one statement, with the `UPDATE` and the `job_executions` insert in the same statement as CTEs.

Decision. Option 3. One statement, in `app/db/sql/claim_jobs.sql`.

Why not option 1. Under `READ COMMITTED`, workers A and B can both `SELECT` job J, both see `status='queued'`, and both `UPDATE` it. The second `UPDATE` does not fail — it overwrites. Two workers execute J, and nothing in the database notices. Taking the row lock *inside the same statement* as the read is what removes the window.

Why not option 2. Plain `FOR UPDATE` makes worker B **block** on the row worker A is claiming, then re-read it after A commits — B waits for work it provably cannot have, and claim latency across the fleet serialises on the slowest claimer. `SKIP LOCKED` steps over locked rows and takes the next available ones, so K workers partition the ready set with no coordination and no blocking.

Consequences. - Claim, transition, and execution-row insert are one round trip and one transaction. - The statement is raw SQL in a `.sql` file, so it can be pasted into `psql` and `EXPLAIN`ed. - `COALESCE(max(n), 0)` guards the `LIMIT`: if the queue is paused or missing, the headroom CTE is empty, a bare scalar subquery yields `NULL`, and **`LIMIT NULL` in Postgres means no limit** — a paused queue would drain itself completely. `max()` over an empty set is `NULL`, coalesced to `0`, and `LIMIT 0` claims nothing. - Evidence: `test_concurrent_claims_are_disjoint` — 100 jobs, 8 concurrent sessions, union of claimed ids is 100 and the pairwise intersection is empty.

Revisit if. The claim's `EXPLAIN` stops using `ix_jobs_claim`, or per-queue claim contention shows up in `pg_stat_activity` as a wait bottleneck.

ADR-003 — Exact queue concurrency caps

Context. `queues.max_concurrency` must cap how many jobs from a queue run at once, across the whole fleet.

Options. 1. Count in-flight jobs, then claim up to the difference (no lock). 2. Maintain an `active_count` counter column, incremented at claim and decremented at completion. 3. A `queue_slots` table with one row per slot, claimed with `SKIP LOCKED`. 4. Lock the `queues` row with `FOR UPDATE`, compute headroom inside the claim statement. 5. Lock the `queues` row with **`FOR NO KEY UPDATE`**, compute headroom inside the claim statement.

Decision. Option 5.

Why not option 1 — this is the one that looks fine and is not. It is check-then-act. With `max_concurrency=10` and 8 in flight, workers A and B each read `8` in their own snapshot, each compute a budget of 2, and each claim 2 *disjoint* rows. `SKIP LOCKED` guarantees disjointness, which is the wrong invariant here — twelve jobs now run against a cap of ten. With K workers the overshoot is `(K−1) × batch_size`, and it grows exactly when the system is busiest.

Why not options 2 and 3. A counter drifts the moment a worker dies between decrementing and committing, and drift needs a reconciliation sweep that itself needs testing. A slots table is an extra table, an extra index, and an extra thing to leak.

Why `FOR NO KEY UPDATE` and not `FOR UPDATE`. This is the subtle half. Inserting a job takes a **`FOR KEY SHARE`** lock on the referenced `queues` row, via the foreign key. `FOR UPDATE` conflicts with `FOR KEY SHARE` — so every claim would block every enqueue on that queue for the duration of the claim, turning a throughput optimisation into a producer stall that only appears under load. `FOR NO KEY UPDATE` still conflicts **with itself**, so claimers remain mutually exclusive (the invariant that matters), but it does **not** conflict with `FOR KEY SHARE`, so producers are never blocked.

Consequences. - The cap is **exact**, not approximate, and trivially testable. - Claim throughput *per queue* is serialised — the honest cost. The lock is held for a single short statement; different queues never contend; horizontal scale comes from more queues plus a larger `batch_size`. - Evidence: `test_claim_respects_max_concurrency` — cap 3, 12 concurrent claimers, 50 jobs, in-flight never exceeds 3 at any sample.

Revisit if. A single queue needs more claim throughput than one serialised statement provides. The documented next step is hashing `queue_id` into N shard rows so claims spread across shards while each shard keeps its exact cap.

ADR-004 — Fencing token

Context. A worker that is merely *slow* — a GC pause, a `SIGSTOP`, a network partition — stops heartbeating while its threads keep running. The reaper cannot distinguish this from death, reclaims the job, and a second worker starts it. Both are now executing, and the first one is about to write its result.

Options. 1. Trust `lease_expires_at > now()` in the completion predicate. 2. Use the existing `lock_version` optimistic-locking column. 3. A dedicated `lease_epoch bigint`, bumped on **every** ownership change.

Decision. Option 3.

Why not option 1 — the ABA hole. Worker W1 claims job J, stalls, is reaped, and then **re-claims J itself** while its original zombie coroutine is still alive. The zombie's completion matches `worker_id = W1` *and* a live `lease_expires_at`. It commits a stale result as the current attempt's outcome, and every predicate passes. An epoch counter does not repeat, so the zombie's `lease_epoch` can never match again.

Why not option 2. `lock_version` is trigger-owned and bumped on *every* update, heartbeats included. A heartbeat from the legitimate owner would invalidate the owner's own completion. It can serve ORM optimistic locking; it can never be the fence.

Decision detail. Every worker write carries the epoch it claimed under:

```
WHERE id = $1 AND worker_id = $2 AND lease_epoch = $3 AND status = 'running'
```

Zero rows updated means the lease was stolen. The worker discards its result, logs an `abandoned` event, and does not retry the write.

Consequences. - Stale writes are impossible at the row level, under any interleaving. - **Residual risk, stated plainly: fencing protects the database row. It cannot un-send an email the zombie already sent.** That is the handler's contract — ADR-005. - Evidence: `test_stolen_lease_write_is_fenced` and `test_zombie_after_same_worker_reclaim_rejected` (the ABA case specifically).

Revisit if. Never, for this mechanism. If lease ownership ever becomes shared (multiple runners per job), the fence needs to become per-runner rather than per-job.

ADR-005 — Delivery semantics

Context. What does the system promise a handler author?

Options. 1. Claim exactly-once semantics and hope. 2. Claim at-least-once, and give handlers a written contract. 3. Implement a two-phase commit protocol with handlers.

Decision. Option 2. **This system is at-least-once, not exactly-once.**

Why. Exactly-once *delivery* across a process boundary is impossible. The worker cannot atomically commit "job done" to Postgres and "side effect performed" to a third-party API — they are different systems with different transactions. Every protocol has a window where one succeeded and the other did not. A system that claims exactly-once is a system whose author has not found the window yet.

What is actually guaranteed: - At most one worker holds a valid lease on a job at any instant. - At most one execution can record a result for a given `lease_epoch`. - A handler is invoked **at least** once, and may be invoked more than once under partition.

The handler contract. Handlers receive `job_id` and `attempt` on `JobContext` and **must be idempotent**: key external writes on `job_id`, or make the operation naturally idempotent (upsert, not insert). This is documented, and the seeded demo handlers are written this way.

Note that this is *execution-level* idempotency, and it is a different problem from the `Idempotency-Key` header, which is *request-level* — it stops one HTTP retry creating two jobs. Both exist; they solve different failures.

Consequences. Honest, testable guarantees. The failure matrix in ARCHITECTURE-adjacent docs lists the residual risk for each failure rather than claiming none.

Revisit if. A use case genuinely requires effectively-once. The path is transactional outbox on the handler's side, not a change here.

ADR-006 — Retry destination

Context. A failed attempt with attempt budget remaining has to become eligible again, later.

Options. 1. `status = 'queued'` with a future `run_at`, and add `run_at <= now()` to the claim predicate. 2. `status = 'scheduled'` with a future `run_at`, promoted by the scheduler when due.

Decision. Option 2.

Why not option 1 — it silently deletes backoff. The claim query has **no `run_at` predicate** — that is what keeps `ix_jobs_claim` narrow and its partial index small. A `queued` job is by definition due. Put a backoff retry into `queued` and the very next poll claims it ~500ms later: five attempts burn in about 20ms, the backoff table becomes decorative, and the system hammers the dependency that is already down at maximum rate. The bug is invisible in a unit test and obvious in production.

Adding `run_at <= now()` to the claim would fix it, at the cost of widening the hot index and making every claim scan rows it cannot have.

Consequences. - One promotion path for delayed jobs, cron occurrences, and retries — the same code, tested once. - A dead promoter stalls all three. That is why `system_state.last_run_at` is on the dashboard (ARCHITECTURE §5); it is the system's most dangerous silent failure. - Evidence: `test_retry_goes_to_scheduled_not_queued` — a failed attempt with budget left lands in `scheduled` with `finished_at` NULL and the fence advanced; a claim then returns nothing (the predicate is `status='queued'`, full stop), and with `run_at` pushed an hour out the promoter returns nothing either. Both halves are asserted, which is what makes it a test of the *destination* rather than of the wording of a status.

Revisit if. Promotion latency becomes the dominant term in end-to-end retry time. `LISTEN/NOTIFY` would replace the promoter's poll.

ADR-007 — Where `attempt` is incremented

Context. `attempt` drives backoff and the dead-letter decision, so where it increments changes what a graceful shutdown has to do.

Options. 1. Increment at claim, and decrement when a claimed-but-unstarted job is released. 2. Increment at the `claimed → running` transition.

Decision. Option 2.

Why. A job released from `claimed` **provably never executed** — the handler is invoked only after the guarded start returns a row. So there is nothing to decrement, graceful release costs nothing, and `attempt` is **only ever incremented**, by exactly one statement, on exactly one edge. Nothing in the schema enforces that monotonicity — there is no transition trigger and no CHECK on `attempt`'s direction; it holds because no other statement writes the column. Option 1 needs a compensating write on a shutdown path — the one path least likely to run to completion, since it executes precisely when the process is being killed.

Consequences. - The start statement must be a guarded write **whose rowcount is checked**:

```
sql UPDATE jobs SET status='running', started_at=now(), attempt=attempt+1, updated_at=now()
WHERE id=$1 AND worker_id=$2 AND lease_epoch=$3 AND status='claimed' RETURNING id;
```

If this returns zero rows the shutdown path already released the job, and the executor **must cancel the task before invoking the handler**. Skipping that check is a double-execution bug on every deploy: the release commits while the start is in flight, another worker claims the now-`queued` job immediately, and the original handler runs to completion anyway. No lease expiry, no partition, no slow worker involved — just a rolling restart. - Evidence: `test_sigterm_releases_unstarted_claims`, `test_start_rowcount_zero_means_do_not_run`.

Revisit if. Never; the alternative is strictly worse.

ADR-008 — Full jitter

Context. When a shared dependency fails, every in-flight job fails at nearly the same instant and schedules its retry from nearly the same instant.

Options. 1. Deterministic backoff, no jitter. 2. Narrow jitter, e.g. ±10%. 3. Full jitter: `delay = random() × capped_delay`.

Decision. Option 3, applied to `fixed` (`base`), `linear` (`base × n`), and `exponential` (`base × 2^(n−1)`), each capped at `backoff_max_ms` first.

Why. Deterministic backoff reproduces the stampede at every tier — the thundering herd arrives again at t+1s, t+2s, t+4s, each time in lockstep, each time knocking over a dependency that was halfway up. Narrow jitter merely smears the spike; the herd is still a herd. Full jitter spreads retries uniformly across the whole window, which is what actually lets a recovering dependency come back.

Consequences. - Retry timing is non-deterministic, so tests assert **bounds**, not equality: `test_backoff_sequence_per_strategy` checks each delay lies within `(0, capped]` and that the cap holds. - Mean delay is half the deterministic value, which is the intended trade: the tail matters, the mean does not.

Revisit if. A queue needs a strict minimum inter-attempt gap (rate-limited third-party APIs). Decorrelated jitter with a floor would be the replacement.

ADR-009 — Manual retry is a new job

Context. "Retry failed jobs" is a core requirement, and the DLQ needs a replay action.

Options. 1. Mutate the terminal job back to `queued` and reset its counters. 2. Insert a new job with `replay_of_job_id` pointing at the original.

Decision. Option 2.

Why. Option 1 gives terminal states out-edges — `dead_letter → queued` and `completed → queued` would have to become legal — and once those edges exist, nothing distinguishes a deliberate replay from a bug that resurrects a completed job. That matters more here than it would elsewhere, because the state machine is enforced only by each statement's `WHERE` clause naming its source status (`DATABASE.md` §5): there is no trigger holding the diagram, so an edge that becomes writable is an edge anything can take. It also destroys history: the attempt history, the error, the timings and the DLQ record are all overwritten by the replay, so the one artefact an operator needs to understand *why* it failed is gone.

Consequences. - Terminal states have **zero out-edges**, so the diagram in `DATABASE.md` §5 stays small enough to read in one sitting. - The UI timeline can show the original and the replay as separate, linked runs. - `ux_jobs_live_idempotency` is scoped to *live* statuses, so replaying a dead-lettered job that carried an `Idempotency-Key` does not collide with its own ancestor. **This is not covered by a test** — see `TESTING.md` §3. The index is right; the assertion is missing. - Job count grows with replays. That is what a history table is for.

Revisit if. Never.

ADR-010 — Keyset pagination

Context. The job explorer pages through a table that is being inserted into constantly.

Options. 1. `LIMIT` / `OFFSET` with a `total` count. 2. Keyset on the `(created_at, id)` tuple, cursor encoded base64.

Decision. Option 2.

Why. `jobs` is insert-heavy by design. With offset pages, rows inserted between page 1 and page 2 shift the window: the reader **skips** rows and **sees duplicates**, and neither is visible as an error. Deep offsets also make the database scan and discard everything it skips. Keyset reads from the cursor forward, so a concurrent insert cannot displace a page.

Consequences. - `total` is deliberately absent from cursor pages — an exact count over a live table costs a scan and is stale before it is rendered. Counts come from `GET /queues/{id}/stats`, which reads the partial `ix_jobs_depth`. - The cursor tuple must match index order exactly, which is why `ix_jobs_project_created` and `ix_jobs_queue_status_created` are `(… created_at DESC, id DESC)`. - No page-number UI. Infinite scroll / "load more" instead. - **No test covers this.** A cursor walked to exhaustion under concurrent inserts returning each job exactly once is the property that matters, and it is unasserted — see `TESTING.md` §3.

Revisit if. A user-facing requirement genuinely needs jump-to-page.

ADR-011 — Primary key types

Context. Identifier choice affects index locality, enumerability, and payload size on the fastest-growing tables.

Options. 1. `bigint` identity everywhere. 2. UUIDv4 everywhere. 3. UUIDv7 for entities, `bigint` identity for append-only children.

Decision. Option 3.

Why. UUIDv4 is random, so every insert dirties a different index leaf — write amplification and poor cache locality on exactly the table under the most insert pressure. UUIDv7 is time-ordered, so inserts stay on the rightmost page, while keeping the properties that matter externally: non-enumerable (a `bigint` job id leaks volume and lets a client walk the id space) and client-generable before the round trip. For `job_executions`, `job_logs`, `worker_heartbeats` and `queue_stats_minute` none of that applies — they are never referenced from outside — so 8 bytes beats 16 on the rows there are most of.

Consequences. - Ids are generated in the application (`uuid-utils`), not by the database, so a client can construct a job id before it POSTs. - Mixed id types across tables. The per-table mapping is pinned in `SCHEMA_NAMES.md` §10 so it is never a guess.

Revisit if. Postgres ships a native `uuidv7()` — then generation can move server-side, though client-generability would be lost.

ADR-012 — The scheduler is its own process

Context. Something must promote due jobs, dispatch cron occurrences, reap expired leases, sweep dead workers, and enforce retention.

Options. 1. FastAPI background task inside the API process. 2. A leader-elected role inside the worker fleet. 3. A separate `scheduler` process, made safe under N replicas by database constraints.

Decision. Option 3.

Why not option 1. It ties job liveness to HTTP traffic — an idle API stops promoting delayed jobs and stops reaping leases, so the system degrades exactly when nobody is watching. And it double-fires on every API replica, so scaling the API multiplies the scheduler.

Why not option 2. Leader election is a distributed-systems problem this system does not need to solve, with its own failure modes (split brain, fencing of the *leader*, lease renewal) that are strictly harder than the problem being solved.

Why option 3 is safe under N. Correctness comes from the database, not from singleton-ness: - Cron double-promotion is impossible — `ux_jobs_schedule_occurrence` on `(schedule_id, scheduled_for)`. - Reaper and promoter overlap is harmless — every statement is `FOR UPDATE SKIP LOCKED`, so two schedulers partition the work rather than collide.

`pg_try_advisory_lock` is used **only** to stop redundant ticks burning CPU. If it is never acquired, the system is still correct — which is the test of whether a lock is load-bearing.

Consequences. - A third process to run and to document. The README says loudly that **nothing executes without a worker**, and the scheduler's staleness is on the dashboard via `system_state`. - Evidence: `test_cron_two_schedulers_one_job_per_occurrence`, `test_expired_lease_reclaimed_once`. - **Lock ordering is fixed at `jobs → workers`**, and the dead-worker sweep runs in its **own transaction**, separate from lease reclaim. Combining them — holding `jobs` locks while taking `workers` locks — deadlocks against a heartbeat that took `workers` first and then blocked on `jobs`. The deadlock victim is preferentially the lagging worker, which manufactures the very lease expiry the design is trying to avoid.

Revisit if. Tick frequency needs to exceed what polling can serve; `LISTEN/NOTIFY` is the seam.

ADR-013 — Tenant isolation without RLS

Context. Multi-tenancy is a hard requirement. A cross-tenant read is the worst bug this system could ship.

Options. 1. Filter by `organization_id` in each query by hand, over composite FKs that make cross-tenant rows unrepresentable. 2. Option 1 **plus** a `TenantSession` that injects the predicate automatically via SQLAlchemy's `with_loader_criteria`. 3. Option 2 **plus** Postgres Row-Level Security.

Decision — and what actually shipped is option 1, not option 2. This ADR originally recorded option 2 as the decision. It is worth correcting in place rather than quietly, because the gap between the two is the difference between a control and a convention.

What is real. The structural half, and it is the stronger half:

- `jobs` declares `UNIQUE (id, organization_id)` (`uq_jobs_id_organization_id`).
- `job_executions` references `(job_id, organization_id)` rather than `job_id` alone (`fk_job_executions_job`).

A child row whose tenant disagrees with its parent's is therefore **not representable**. The database rejects it regardless of what the application does, and this holds for the raw-SQL hot path exactly as it holds for the ORM. A forged cross-tenant row is impossible, not merely unlikely.

What is not real: the loader hook. `TenantSession` and `with_loader_criteria` are **zero hits in the codebase**. Every tenant-scoped read filters by hand — each router writes `.where(… .organization_id == principal.organization_id)` explicitly — which is precisely the option this ADR once dismissed as "a policy, not a control". That dismissal was correct, and the honest position is that the query half of this design is a convention that relies on nobody forgetting, on a codebase that will grow.

Why it matters, stated exactly. The composite FKs stop a cross-tenant row being *written*. They do nothing to stop a router that omits its predicate from *reading* across tenants, and nothing would catch it: there is no loader hook, no RLS, and no test asserting that every tenant-scoped query emits an `organization_id` predicate. For a system whose worst possible bug is a cross-tenant read, that is the weakest link in the design, and it is one file of SQLAlchemy event wiring away from being closed. The loader hook is the next thing to build here, ahead of RLS.

Why not option 3 — a real trade-off, not an oversight. RLS is a *second* enforcement layer, and adding it before the *first* behavioural layer exists gets the order wrong. It also costs roughly a day, and every fixture, migration or admin script that forgets to `SET` the GUC turns into an opaque "my test returns zero rows and no error" debugging session. Spending that day on the reliability core was the better trade; spending the next hour on the loader hook is a better trade still.

The documented hardening step, so this is a decision with a migration path rather than a gap:

```
ALTER TABLE jobs ENABLE ROW LEVEL SECURITY;
ALTER TABLE jobs FORCE ROW LEVEL SECURITY;

CREATE POLICY jobs_tenant_isolation ON jobs
    USING      (organization_id = current_setting('app.current_org', true)::uuid)
    WITH CHECK (organization_id = current_setting('app.current_org', true)::uuid);

-- Set once per checked-out connection, inside the transaction:
SELECT set_config('app.current_org', $1, true);
```

Repeat per tenant-scoped table. The `true` third argument to `current_setting` returns `NULL` rather than raising when the GUC is unset, so an unset connection sees zero rows instead of an error — which is exactly the opaque failure described above, and why the rollout needs a session-level assertion that the GUC is set.

Consequences. Enforcement is asymmetric, and the asymmetry should be understood rather than averaged: **structurally strong** (the FK graph, which no code path can bypass) and **behaviourally weak** (hand-written predicates, unenforced and untested). A raw-SQL statement that bypasses the ORM must carry its own `organization_id` predicate — the hot-path `.sql` files do, and they are reviewed as SQL for exactly this reason. The same is spelled out from the schema's side in `DATABASE.md` §8.

Revisit if. A compliance requirement demands defence in depth at the database layer, or raw-SQL surface grows beyond the reviewed hot path.

ADR-014 — Two roles, not four

Context. The spec lists RBAC as a **bonus**.

Options. Two roles (`owner` , `member`); or the full `owner/admin/operator/viewer` ladder.

Decision. Two roles, via a single `require_member` dependency on mutating routes. Reads require authentication plus org membership.

Why. A four-tier ladder puts a *bonus* on the critical path of every endpoint: every route needs a permission decision, every route needs a test per tier, and the matrix is 4 × routes. That is budget taken directly from the reliability core.

The deferred design, ready to implement: `viewer` = read-only; `operator` = viewer + pause/resume, cancel, replay; `admin` = operator + queue and schedule CRUD, member invite; `owner` = admin + billing, member removal, org delete. It slots into the same dependency as a `require_role(min_role)` comparison against an ordered enum — no schema change beyond widening the `role` CHECK.

Consequences. - **Cross-tenant access returns 404, not 403.** A 403 confirms the resource exists, which is an enumeration oracle across tenants. This is how the routers behave; **no test asserts it** (`TESTING.md` §3). - There is **no last-owner guard** — no constraint trigger, no application check. Nothing can currently demote or remove the last owner because the membership mutation endpoints are not built (`API.md` §1), so the hole is closed by absence rather than by a control. The guard lands with the endpoint that needs it, and it has to land in the same change.

Revisit if. Real multi-user orgs need finer separation of duty.

ADR-015 — Polling, not WebSockets

Context. The dashboard must show job progress that changes second by second.

Options. WebSockets / SSE push; or TanStack Query polling with per-view intervals.

Decision. Polling.

Why. WebSockets are listed as a **bonus** in the spec. The connection lifecycle — reconnect with backoff, auth on upgrade, resubscribe after reconnect, server-side fanout, and the "did I miss an event while disconnected" reconciliation — is a substantial amount of code whose failure modes are subtle and whose marks are zero. Polling is correct by construction: every poll is a fresh read of the truth, and a missed poll self-heals.

The cost is bounded and stated:

Query	Interval
Job detail (non-terminal)	2s — **stops on terminal status**
Job list	5s
Queue health	10s
Workers	10s
Throughput chart	30s
Anything on a hidden tab	paused

Terminal-stop and hidden-tab-pause are what keep an open dashboard from being a load generator.

Consequences. Up to one interval of staleness. The polling hooks are the seam: swapping the transport later touches the hooks and nothing else.

Revisit if. Interval load becomes material, or sub-second latency is required.

ADR-016 — Inline Mermaid diagrams

Context. The architecture diagram and ER diagram are required deliverables.

Options. Exported images; separate `.mmd` files plus a drift checker; inline fenced Mermaid in the Markdown.

Decision. Inline fenced Mermaid.

Why. GitHub renders it natively, so a reviewer sees the picture without cloning. It is **plain text in the diff**, so a schema change and its diagram change are visible in the same review. A parallel `.mmd` file is a second copy of the same truth, which means it can drift, which means building a drift checker — a tool to protect against a problem created by the file layout.

Consequences. No binary assets in the repo. The diagram cannot be styled beyond what Mermaid offers, which is fine.

Revisit if. A diagram outgrows Mermaid's expressiveness.

ADR-017 — `READ COMMITTED`

Context. Postgres offers `READ COMMITTED`, `REPEATABLE READ`, and `SERIALIZABLE`.

Options. Raise the isolation level for the claim path, or keep the default.

Decision. Keep `READ COMMITTED`, the default, everywhere.

Why. The invariant the claim needs is **per-row mutual exclusion**, and `FOR UPDATE SKIP LOCKED` provides exactly that regardless of isolation level. `REPEATABLE READ` would add serialisation failures (`40001`) on the hottest statement in the system, which means a retry loop, which means retry-loop bugs — for no additional safety, because the row lock already does the work.

The corollary is that `READ COMMITTED` is *not* sufficient for a `SELECT`-then-`UPDATE` claim (ADR-002); the mitigation is the single-statement lock, not a higher isolation level.

Consequences. No serialisation-failure retry logic anywhere in the codebase. Any future read-modify-write that spans statements must take an explicit row lock, and the reviewer's question for such a patch is "where is the `FOR UPDATE`".

Revisit if. A new cross-row invariant appears that a row lock cannot express.

Testing

```
make test
```

```
make test-concurrency
```

```
make check
```

`pytest` + `pytest-asyncio` + `httpx.AsyncClient`, against a **real local PostgreSQL** database (`cody_test`), with `alembic upgrade head` run once per session.

34 tests across seven files. Nine of those are regression tests in the strict sense — written after a defect was found, and **verified to fail against the code they replace**: two of the three in `tests/test_execution_timing.py`, and all six in `tests/test_worker_lifecycle.py`. That is what separates a regression test from one written to match whatever the code already does, and it is the distinction §3 leans on. 34 is a small number, and it is deliberately spent: it buys the reliability invariants in §3 — disjoint claims, an exact concurrency cap, lease fencing including the ABA case, orphaned-execution recovery, the shutdown race, and cron under two schedulers — rather than breadth across routers. Every one of them runs against genuinely independent **committed** sessions (§2), which is the difference between testing `SKIP LOCKED` and testing nothing. What that choice costs is listed honestly at the end of §3.

---

1. No testcontainers

Docker is not installed on the target machine, so the test suite cannot depend on it. It also should not: the behaviour under test is `FOR UPDATE SKIP LOCKED`, `FOR NO KEY UPDATE` lock conflicts, partial unique indexes, and CHECK constraints. None of that survives being mocked, and none of it is reproducible against SQLite. The tests run against the same PostgreSQL 15 the application runs against, or they prove nothing.

`make setup` creates `cody_test`. Point somewhere else with `CODITY_DATABASE_URL=postgresql+asyncpg://…/other_db`.

---

2. One mode: real commits, cleaned between tests

There is no rollback fixture and no two-fixture split. `backend/tests/conftest.py` defines five fixtures, and every test in the suite gets the same treatment:

Fixture	Scope	What it does
`_migrate`	session, autouse	`alembic upgrade head` once against `codity_test`
`engine`	function	An async engine on **`NullPool`**, so every session gets its own connection
`sessionmaker_`	function	`async_sessionmaker(engine, expire_on_commit=False)`
`_clean`	function, **autouse**	Teardown: every table is emptied, identities restarted, cascading
`_reset_app_engine`	function, autouse	Clears the module-level engine cache in `app.db.session`

This is the decision that makes the concurrency tests mean anything, and it is why the faster option was rejected rather than overlooked. A connection-bound transaction rolled back after each test is the standard pytest pattern and it is measurably quicker — but **`SKIP LOCKED` semantics do not exist inside a single transaction.** A transaction cannot skip its own locks, two "workers" sharing one session are one worker with extra steps, and every concurrency assertion in §3 would pass *vacuously* while testing nothing at all. `NullPool` plus committed writes plus a teardown that empties the tables is what buys genuinely independent sessions. The cost is teardown time, paid once per test, and it is the right trade for a suite whose entire point is what happens *between* connections.

`_reset_app_engine` exists for a narrower reason, worth knowing before you debug it: `app.db.session` caches an engine at module level, pytest-asyncio gives each test a fresh event loop, and a cached engine holds connections bound to a dead loop — so the *next* test dies with `Event loop is closed`.

Concurrency tests are marked `@pytest.mark.concurrency`, applied at **module level** in `test_concurrency.py`, `test_retry.py`, `test_scheduling.py`, `test_execution_timing.py` and `test_worker_lifecycle.py`. So `make test-concurrency` (`pytest -m concurrency`) selects **29 of the 34 tests** — most of the suite, not a corner of it. The five it deselects are the three migration tests and the two API end-to-end tests.

---

3. Critical tests

These are the evidence for the reliability design. **They are not cut under time pressure.**

Claiming and concurrency

Test	Arrange → Act → Assert
`test_concurrent_claims_are_disjoint`	100 queued jobs, 8 concurrent sessions → all claim → union == 100, pairwise intersection empty
`test_claim_respects_max_concurrency`	cap 3, 12 concurrent claimers, 50 jobs → claim → in-flight never exceeds 3 at any sample
`test_claim_cap_holds_when_a_claimer_waits_on_the_queue_lock`	the two-session deterministic form: A fills the cap and holds its transaction open, B blocks on the queue-row lock → after release, B claims **zero**, in-flight is still 3
`test_pause_stops_claiming_resume_restarts`	pause mid-drain → claims return 0; resume → claims resume
`test_claim_uses_partial_index`	`EXPLAIN` the real claim → references `ix_jobs_claim`, no `Seq Scan`, no `Sort`

`test_claim_cap_holds_when_a_claimer_waits_on_the_queue_lock` is the one that found a real bug. Locking the queue row does *not* give the rest of a single statement a fresh snapshot: under `READ COMMITTED` a statement keeps the snapshot it took when it started, and releasing a row lock re-checks only the *locked row*. A claimer that waited behind another transaction's commit therefore computed its headroom from a pre-lock count — the exact check-then-act race the lock exists to close, merely serialised. The fix is why the lock is `lock_queue.sql`, issued as its **own statement** before `claim_jobs.sql` in the same transaction: a new statement gets a new snapshot, taken once the lock already makes it safe to act on.

Leases, fencing, recovery

Test	Arrange → Act → Assert
`test_expired_lease_reclaimed_once`	claim, expire lease, two concurrent reapers → exactly one requeue, `lease_epoch` +1
`test_stolen_lease_write_is_fenced`	claim at epoch e, reap, re-claim → complete with epoch e → rowcount 0, job untouched
`test_zombie_after_same_worker_reclaim_rejected`	W1 claims, is reaped, W1 re-claims (epoch e+2), zombie completes with e → rejected — **the ABA case**
`test_orphan_execution_does_not_block_next_attempt`	claim, lose the worker, reap, claim again → the second execution insert succeeds

`test_orphan_execution_does_not_block_next_attempt` guards the `closed` CTE in `reap_leases.sql`. `ux_job_executions_open_one` permits one open execution per job; if an abrupt worker death leaves an orphaned open row and the reaper does not close it, the *next* claim's execution insert raises `23505` — and so does every claim after that. The job becomes permanently unexecutable, and it is precisely the flagship crash-recovery demo that wedges it.

Retries and the DLQ

Test	Arrange → Act → Assert
`test_backoff_sequence_per_strategy`	attempts 1..5 per strategy → `run_at` deltas within jitter bounds and capped at `backoff_max_ms`
`test_retry_goes_to_scheduled_not_queued`	fail with budget remaining → status `scheduled`, not claimable until `run_at`
`test_exhausted_job_dead_letters_once`	`max_attempts=2`, always-fail handler → exactly one `dead_letter_entries` row, `finished_at` set

`test_backoff_sequence_per_strategy` is parametrised over `fixed`, `linear` and `exponential`, so it is three of the 28. Backoff is **full jitter**, so these assert bounds, never equality. A test that asserts an exact delay against a jittered schedule is a flaky test that will be deleted by whoever inherits it.

Not tested: that replaying a dead-lettered job which carried an `Idempotency-Key` succeeds without a unique violation. `ux_jobs_live_idempotency` is scoped to live statuses precisely so that it does, and ADR-009 rests on it, but no test asserts it. It is the most obvious gap in this tier.

Shutdown

Test	Arrange → Act → Assert
`test_sigterm_releases_unstarted_claims`	SIGTERM with buffered claims → jobs return to `queued`, `attempt` unchanged, nothing lost
`test_released_claim_can_be_claimed_again`	graceful release → the execution row is **closed**, and the next worker can claim the job
`test_start_rowcount_zero_means_do_not_run`	release the job between claim and start → **the handler is never invoked**, and no attempt is consumed

The second is the deploy-time double-execution bug: the release commits while the start is in flight, another worker claims the now-`queued` job, and the original handler runs to completion anyway. No lease expiry and no partition required — just a rolling restart.

Scheduling

Test	Arrange → Act → Assert
`test_cron_two_schedulers_one_job_per_occurrence`	2 schedulers, one due schedule → exactly one job for that `scheduled_for`
`test_cron_advances_when_insert_suppressed`	pre-insert the occurrence, tick → `next_occurrence_at` still advances
`test_cron_does_not_materialise_into_a_paused_queue`	tick a due schedule on a paused queue → no job, `skipped_occurrences` increments

The second guards a schedule that dies silently. If `next_occurrence_at` is advanced only when the insert returns a row, a suppressed insert (a prior tick that committed the job then crashed, or an operator's "run now") freezes the cursor: the schedule re-selects, re-conflicts, and does nothing — forever, once per second, with no error anywhere.

Execution timing — `tests/test_execution_timing.py`

The newest file, and the only one written **after** the bug it describes was found rather than before.

Test	Arrange → Act → Assert
`test_start_job_marks_the_attempt_running`	claim → the execution row is `claimed` with `started_at` NULL; start → it is `running` with `started_at` set
`test_completed_attempt_records_a_duration`	claim, start, wait, complete → `duration_ms` is **not NULL** and ≥ 0
`test_start_job_does_not_touch_a_stale_epochs_attempt`	start with a superseded `lease_epoch` → neither the job row nor the execution row moves

Two of these three were verified to fail against the pre-fix code, and that is what makes them regression tests rather than decoration. `start_job` updated only `jobs`; the execution row opened at claim time kept `started_at` NULL forever. Both `complete_job` and `fail_job` derive `duration_ms` from `e.started_at`, so **every duration in the product was NULL**, and `ExecutionStatus.RUNNING` was unreachable — an attempt could never be observed in progress, and the job timeline jumped straight from `claimed` to a terminal state.

The reason it survived is worth naming: the existing end-to-end test *selected* `duration_ms` and never asserted on it. A test that reads a column without checking its value proves the query compiles and nothing else. These assert the value.

The third test does not reproduce a past bug — it pins the fence. The new execution `UPDATE` had to be guarded on `lease_epoch` exactly as the `jobs` `UPDATE` is, or the fix would have opened a fresh hole beside the one it closed.

Worker lifecycle — `tests/test_worker_lifecycle.py`

Six tests, each reproducing an interleaving only a **second** worker can reach, and each one written against a defect it reproduces rather than against current behaviour.

Test	Arrange → Act → Assert
`test_unregistered_release_cannot_close_a_live_execution_row`	a stale worker wakes after being reaped and releases → it must not close the *successor's* live execution row
`test_unregistered_release_still_closes_its_own_attempt`	the same release → its own attempt row is closed, so the next claim can open one
`test_unregistered_release_parks_the_job_instead_of_requeueing_it`	release a job whose handler is unregistered → parked, not returned to `queued` for the next poll to re-claim
`test_unregistered_count_dead_letters_the_job`	repeated unregistered releases → bounded by `UNREGISTERED_MAX_RELEASES`, then dead-lettered
`test_beat_follows_the_held_lease_not_the_claimable_queues`	heartbeat interval sized from leases **held**, not queues subscribed to
`test_pausing_a_queue_cannot_widen_the_beat_mid_flight`	pause a short-lease queue with a job running on it → the beat does not slow below that job's lease

The first three are one bug each in the unregistered-handler release path: an execution `UPDATE` fenced on nothing, so a reaped worker could close a live row belonging to its successor; a release into `queued` that the next ~500ms poll re-claimed, making the claim/release loop unbounded; and an `unregistered_count` that nothing read, so the bound it existed to enforce did not exist. The last two are one bug in heartbeat sizing: the interval came from the queues a worker might claim *from* rather than the leases it actually *holds*, so pausing a short-lease queue starved the heartbeat of a job already running on it — a self-inflicted lease expiry.

Like every other concurrency test here, these drive the real SQL through the real service wrappers on independent committed sessions; the helpers are imported from `test_concurrency.py` rather than re-implemented.

API end-to-end and migrations

Test	Arrange → Act → Assert
`test_posted_job_is_claimed_and_completed`	`POST` a job over HTTP, run a real worker against it → `completed`
`test_paused_queue_is_not_claimed`	pause, `POST` , run the worker → the job stays `queued`
`test_upgrade_creates_schema`	`upgrade head` on an empty database → the expected tables exist
`test_upgrade_downgrade_roundtrip`	empty DB → `upgrade head` → `downgrade base` → clean
`test_enum_labels_are_lowercase_values`	every Postgres enum label matches the Python `StrEnum` value

What is not tested here, stated rather than implied. There is no test asserting that a cross-tenant request returns `404` rather than `403`, no test asserting the error envelope's shape per router, and no test walking the AST to enforce the layering rule in `ARCHITECTURE.md` §3. `LEGAL_TRANSITIONS` in `app/domain/enums.py` is likewise asserted by nothing — there is no status-transition trigger for it to be compared against, because the schema has no triggers at all (`DATABASE.md` §5). Each of those behaviours is implemented; none of them is pinned. The reliability core got the test budget, and this is where the bill came due.

---

4. No coverage gate

`make cov` prints a coverage report. There is deliberately **no `--cov-fail-under`**.

Coverage percentage earns zero marks and does not measure whether the reliability invariants hold. A gate added late in a build creates pressure to write filler serializer tests at exactly the moment the concurrency tests are at risk — optimising the number instead of the property. The tests in §3 are the ones that matter, and they are named individually so their absence is visible.

---

5. Writing a new concurrency test

1. Take `sessionmaker_` . The module-level `pytestmark = pytest.mark.concurrency` already covers the four files that have one; a new file needs its own.
2. Open **separate sessions** — one per simulated worker. Two coroutines sharing a session are one worker with extra steps.
3. Drive them with `asyncio.gather` so the statements genuinely interleave.
4. Assert a **property** (disjointness, a cap, a rowcount of zero), never a timing.
5. Let `_clean` handle teardown — it is autouse and empties every table after each test, so a test that writes outside the fixtures still gets cleaned up. What it cannot clean is anything written to a table not in that list; add it there rather than tidying by hand.

Canonical Names

This file is the single source of truth for every name that appears in more than one place: the database, `app/domain/enums.py`, the OpenAPI schema, and the React client. It was written **before** the code it governs.

Every name drift that has to be reconciled late in a project like this — `status` vs `state`, `dead` vs `dead_letter`, `run_at` vs `scheduled_at` vs `next_run_at`, `is_paused` vs `paused`, which direction priority sorts, whether an id is a `uuid` or a `bigint` — is a symptom of not having this file. Nothing here is negotiable in a pull request; changing a name here is a schema change.

`backend/app/domain/enums.py` is the executable copy. **Import from it. Never redefine a literal.**

---

1. `job_status` — native Postgres `ENUM`

The lifecycle state of a job row. Eight values, no others.

Value	Meaning
`scheduled`	Not yet eligible. Waiting for `run_at`. Covers delayed jobs, cron occurrences, **and backoff retries**.
`queued`	Eligible **now**. This is the only status the claim query looks at.
`claimed`	A worker holds a lease but has not started the handler. `attempt` not yet incremented.
`running`	Handler is executing. `attempt` has been incremented.
`completed`	Terminal. Handler returned.
`failed`	Terminal. Non-retryable failure that is not routed to the DLQ.
`dead_letter`	Terminal. Attempt budget exhausted, or a `PermanentError`. A `dead_letter_entries` row exists.
`cancelled`	Terminal. Cancelled before or during execution.

Terminal set: `completed`, `failed`, `dead_letter`, `cancelled`. Terminal statuses have **zero out-edges** — see `docs/DATABASE.md`. Terminal statuses **require** `finished_at`; the `ck_jobs_terminal_finished` CHECK aborts the transaction if it is omitted.

Not used anywhere: `pending`, `waiting`, `active`, `success`, `error`, `dead`, `retrying`.

2. `execution_status` — native Postgres `ENUM`

The outcome of one *attempt*, recorded on `job_executions`. Seven values.

Value	Meaning
`claimed`	Row opened at claim time. Open (`finished_at IS NULL`).
`running`	Handler started. Open.
`succeeded`	Handler returned. Closed.
`failed`	Handler raised. Closed.
`timed_out`	Exceeded `jobs.timeout_ms` . Closed. Counts as a failed attempt.
`cancelled`	Cancelled mid-flight. Closed.
`lost`	The reaper closed an orphan whose worker never came back. Closed.

`succeeded` , not `completed` — the job says `completed` , the attempt says `succeeded` , and the difference is load-bearing: a job can complete after several attempts that did not succeed.

3. Timestamps — `run_at` is the only eligibility clock

Column	Table	Meaning
`run_at`	`jobs`	**The single eligibility timestamp.** The promoter moves `scheduled → queued` when `run_at <= now()` . Backoff writes a future `run_at` .
`scheduled_for`	`jobs`	**Cron occurrence instant only.** The nominal time the recurrence rule fired. `NULL` unless `schedule_id` is set (`ck_jobs_schedule_occurrence` enforces the biconditional). Never used for eligibility.
`next_occurrence_at`	`job_schedules`	The next instant the dispatcher should materialise. Advanced from the nominal `scheduled_for` , never from `now()` .
`claimed_at` / `started_at` / `finished_at`	`jobs` , `job_executions`	Lifecycle instants.
`lease_expires_at`	`jobs`	When the reaper may reclaim.
`last_heartbeat_at`	`workers`	Denormalised liveness; the hot check.

Not used anywhere: `scheduled_at` , `next_run_at` , `execute_at` , `eta` , `available_at` .

All timestamps are `timestamptz` . There are no naive timestamps in the schema, in Python, or on the wire. **All time originates from `now()` inside Postgres** — no process compares its own clock to a lease, so clock skew between nodes cannot cause a false reclaim.

4. Queue pause — `is_paused` and `paused_at` , always together

```
CONSTRAINT ck_queues_pause_consistency CHECK (is_paused = (paused_at IS NOT NULL))
```

Pause: `is_paused = true, paused_at = now()` . Resume: `is_paused = false, paused_at = NULL` . Writing one without the other is rejected by the CHECK. There is no `paused` , `enabled` , or `active` boolean.

Pause blocks **admission only**: in-flight work finishes, and the cron dispatcher does not materialise occurrences into a paused queue.

5. Priority — smallint, `-100..100`, HIGHER RUNS FIRST

`ORDER BY priority DESC, run_at ASC, id ASC`. Default `0`.

Two distinct columns, and they are not interchangeable:

Column	Meaning
`queues.priority`	**Inter-queue** weight. How often a worker polls this queue relative to its others.
`queues.default_priority`	The default `jobs.priority` for jobs *created on* this queue.
`jobs.priority`	**Intra-queue** ordering within the claim.

The direction is stated in the API field description and in the OpenAPI schema, because "priority 1" meaning "most important" is the other half of the industry and a silent disagreement here inverts the whole queue.

6. Fencing — `lease_epoch`

`jobs.lease_epoch bigint NOT NULL DEFAULT 0`. Incremented on **every** ownership change: claim, reclaim, retry, graceful release. Every write a worker makes carries the epoch it claimed under; zero rows updated means the lease was stolen and the result is discarded.

- `lease_epoch` is **the fence**. Nothing else is.
- `lock_version` exists on `jobs` as a **reserved column for future ORM optimistic locking**. It is set to `0` at insert and **never incremented** — no trigger owns it (the schema has no triggers) and no statement bumps it. It is inert today, and it could **never** be the fence regardless.
- `lease_expires_at > now()` is **not** a fence — it has an ABA hole (see `docs/DESIGN_DECISIONS.md`, ADR-004).

7. `attempt` — monotonic, incremented at `claimed → running`

`jobs.attempt` starts at `0` and is incremented by exactly one in the guarded `start_job` statement. It is **never** decremented — `start_job.sql` is the only statement that writes the column, and it only ever adds one. Monotonicity is a property of having a single writer, not of a database constraint: there is no trigger rejecting `NEW.attempt < OLD.attempt`, because there are no triggers in this schema at all.

A job released from `claimed` provably never executed, so nothing needs decrementing. This is what makes graceful shutdown free.

`job_executions.attempt_number` is the attempt this row represents — written at claim time as `jobs.attempt + 1`, i.e. the attempt this claim is *about to* become.

8. Retry destination — `scheduled`, not `queued`

A failed attempt with budget remaining goes to `status = 'scheduled'` with a future `run_at`. The claim predicate is `status = 'queued'` **only** — it has no `run_at` term, which is what keeps `ix_jobs_claim` small. A queued job is by definition due.

9. Roles

Two roles: `owner`, `member`. Stored on `organization_members.role`. The `owner/admin/operator/viewer` ladder is designed in `docs/DESIGN_DECISIONS.md` (ADR-014) and deliberately not built.

10. Identifier type, per table

UUIDv7, application-generated. Time-ordered so inserts stay on the rightmost index page, non-enumerable, and generable by the client before the round trip:

`organizations`, `users`, `projects`, `queues`, `retry_policies`, `jobs`, `job_schedules`, `job_batches`, `workers`, `dead_letter_entries`.

`bigint GENERATED ALWAYS AS IDENTITY` — append-only, high-volume, never referenced externally, and 8 bytes beats 16:

`job_executions`, `job_logs`, `worker_heartbeats`, `queue_stats_minute`.

Natural / composite keys:

Table	Key
`organization_members`	`(organization_id, user_id)`
`refresh_tokens`	`jti` (uuid)
`system_state`	`name` (text: `promoter`, `cron`, `reaper`, `retention`)

There is no `worker_queue_assignments` table and no `idempotency_keys` table. Queue subscription is a worker CLI argument, and the job row is itself the idempotency record — see `DATABASE.md` §3.

11. Wire vocabulary

Concept	On the wire
Job lifecycle field	`status` (never `state`)
List payload	`{ "data": [...], "page": {...}, "meta": {...} }` (never `items`, never `results`)
Page cursor	`page.next_cursor`, opaque base64 of `(created_at, id)`
Error	`{ "error": { "code", "message", "details", "request_id" } }`
Correlation	`X-Request-ID` header ↔ `meta.request_id` ↔ `jobs.correlation_id`
Idempotency	`Idempotency-Key` request header

Filter query parameter names match column names exactly: `status`, `kind`, `queue_id`, `handler`, `created_after`, `created_before`, `run_after`, `run_before`. Unknown query parameters are a `400 unknown_query_param`, so a typo'd filter fails loudly instead of silently returning everything.

Deliverables Checklist

Required deliverable	Where	Status
Source code with setup instructions	`github.com/Surianandhan/Codity`, `README.md`	Done, quickstart verified live
Architecture diagram	`docs/ARCHITECTURE.md` (Mermaid, rendered below)	Done
ER diagram	`docs/DATABASE.md` (Mermaid, rendered below)	Done
API documentation	`docs/API.md`, 30 live endpoints	Done
Design decisions document	`docs/DESIGN_DECISIONS.md`, 17 ADRs	Done
Automated tests for critical functionality	`backend/tests/`, 34/34 passing; 29 carry the concurrency marker	Done

Repository

Full source, migration history, and commit-by-commit rationale — including the exact interleaving that triggers each concurrency bug found and fixed — are at:

https://github.com/Surianandhan/Codity

Branch `main`, 7 commits, gate green at every commit (ruff, mypy `--strict`, `alembic check`, pytest, frontend build).

The commit history is worth reading as part of the submission. Each message states the failing scenario rather than the change: which interleaving produced a double execution, why `FOR NO KEY UPDATE` is required where `FOR UPDATE` would deadlock against the foreign key's `FOR KEY SHARE`, and why a claim returning zero rows must cancel the task rather than proceed.